

MIAMA-HUB developer reference

Contents

MIAMA-HUB	2
README guide	2
1. Architecture and API	2
Package responsibilities	2
Core objects	3
End-to-end flow	3
Profile objects and the API boundary	3
R6 Hub method outline	4
Configuration versus appraisal request	5
2. Tab 1: Set up the appraisal	5
Geographic levels and options	5
Timeframe value conversion	6
Appraisal summary values	6
3. Runtime configuration and source loading	7
Current development workflow for <code>appraisal_inputs</code>	7
Configuration orientation	7
Request-driven source loading	7
4. Tabs 2-4: Specify and refine active travel	7
Editable assumptions: complete, do not replace, user inputs	8
Reference UI value extraction	8
HUB session state and refresh behavior	11
Spread plot payloads and slider redistribution	12
5. Build the counterfactual and health effects	17
Reference appraisal scope	17
Initialize counterfactual data	18
Feasibility rules and sampling sources	18
Users: derive counterfactual number of active mode users	20
Trips: derive counterfactual number of active mode trips	20
Canonical conversion of alternative Tab 2 volume inputs	22
Counterfactual health outcomes	23
6. Tab 5: Present and export results	25
Appraisal summary population	25
6.1 Health outcome units and aggregation	25
Tab 5 health outcome choices	28
Shared UI option catalogues	28
Tab 5 UI integration lifecycle	28
Tab 5 result exports	33
Appraisal report: inputs, results and assumptions	34
7. Data packaging, deployment, and performance	36
Synthetic population parquet conversion	36
Development integration backlog	36
Runtime data profiles and locations	36
Profiling full-data API performance	39

MIAMA-HUB

For a report-style description of the active-travel, sampling, physical- activity, and health-impact methodology, see [docs/methodology.qmd](#). A concise presentation version is available in [docs/methodology_slides.qmd](#). The documents distinguish implemented, approximate, and planned behavior; this README remains the developer-facing integration reference.

MIAMA-HUB is a standalone R package that acts as the integration layer between MIAMA-UI and MIAMA-HM.

README guide

This developer reference follows the appraisal from setup through results:

1. [Architecture and API](#) explains package ownership, profile objects, and the session-scoped `Hub` interface.
2. [Tab 1: Set up the appraisal](#) covers geography, modes, timeframes, and summary values.
3. [Runtime configuration and source loading](#) explains configuration, request mapping, and efficient data access.
4. [Tabs 2-4: Specify and refine active travel](#) documents reference defaults, staged REF/CF snapshots, population refinement, trip refinement, and spread controls.
5. [Build the counterfactual and health effects](#) describes row sampling, trip changes, MMET exposure, and health lookup.
6. [Tab 5: Present and export results](#) starts with [health outcome units and aggregation](#), then covers result tables, plots, filters, headline metrics, and exports.
7. [Data packaging, deployment, and performance](#) covers packaged profiles, external full data, manifests, and profiling.

For a report-style methodological account, see [docs/methodology.qmd](#). A concise handoff of recently changed UI conditions is in [docs/ui_condition_contract_issue.md](#).

The package is intended to:

- receive `appraisal_inputs` from the UI
- join health-model outputs with synthetic population data
- maintain filtered `reference_data`
- initialize and manipulate `counterfactual_data`
- prepare comparative risk assessment inputs and summaries
- return compact UI-ready values and result objects

1. Architecture and API

Package responsibilities

MIAMA-HUB is a library, not an app. The package should be callable from MIAMA-UI, but also from standalone R workflows when a valid `appraisal_inputs` object is available.

The current scaffold organizes package code into six responsibilities:

- API
- input mapping
- reference data
- counterfactual data
- CRA
- shared utilities

Core objects

The scaffold is built around the following internal objects:

- `appraisal_inputs_in`
- `reference_data_raw`
- `reference_data`
- `reference_ui_values`
- `counterfactual_data`
- `health_impacts`
- `ui_return_payload`

End-to-end flow

1. Receive `appraisal_inputs`
2. Normalize and map inputs into internal request specs
3. Load HM outputs and synthetic population sources
4. Join those sources into `reference_data_raw`
5. Filter to the relevant `reference_data`
6. Extract compact UI reference values
7. Initialize and manipulate `counterfactual_data`
8. Prepare CRA inputs and health impact summaries
9. Build a compact UI return payload

See [inst/workflows/dev_workflow.R](#) for the active development orchestration flow.

Profile objects and the API boundary

For now, the `receive_appraisal_inputs()` function is an internal **R-level API boundary**, not a web API. Its job is to accept the active MIAMA-UI profile object, normalize it, and expose its submitted values to the rest of MIAMA-HUB.

In MIAMA-UI, the active profile is created from `schemes/default.R`:

```
mdata[["profile"]] <- mdata$schemes$default$appraisal_inputs
```

HUB expects that same canonical parameter-list shape. The field contract is:

- `input_value`: value submitted by the user or Shiny input binding
- `is_filled`: TRUE only when `input_value` should be treated as a submitted appraisal value
- `default_value`: UI prefill/default value, including HUB-derived reference values
- `additional_data`: derived metadata attached to a control without changing its selected `default_value`; currently used for age- and PA-category population counts in Tab 3

`default_value` is deliberately not promoted into `input_value`. This prevents reference defaults from being mistaken for user-supplied counterfactual or results inputs.

At the moment, this boundary returns:

- `appraisal_inputs_in`: the normalized canonical object
- `appraisal_input_values`: a developer-friendly plain named list of values
- `reference_request`: the subset of inputs used for reference-data logic
- `counterfactual_request`: the subset of inputs used for counterfactual logic
- `results_request`: the subset of inputs used for output/result logic

This keeps MIAMA-HUB focused on calculation and data transformation, while letting MIAMA-UI remain the source of truth for interactive input definition. The `request` object is therefore a HUB-internal convenience view, not an object that MIAMA-UI needs to manage directly.

R6 Hub method outline

The R6 Hub object is the intended session-scoped interface for MIAMA-UI. It is initialized with runtime configuration only. Profile objects are supplied to UI-facing methods when needed; HUB then stores the active profile, the internal request view derived from that profile, and any intermediate data objects built during the appraisal.

Primary UI-facing methods:

- `get_appraisal_setup_inputs(profile = NULL)`: returns the setup subset of the profile, currently `ui_version`, `ui_input_scope`, `geo_level`, `geo_id`, `modes`, `intervention_type`, and `data_source`.
- `build_reference_profile_defaults(profile = NULL, refresh = FALSE)`: loads and filters synthpop reference data for the selected geography, summarizes reference values, and writes matching values into each profile field's `default_value`.
- `build_refinement_profile_defaults(profile = NULL, seed = 1L, refresh = FALSE)`: applies filled Tab 2 REF and CF inputs to synthpop-only staged snapshots, summarizes those exact scopes, and writes the resulting Tab 3 population and spread values into advanced `default_value` fields. No HM data are loaded.
- `build_trip_refinement_profile_defaults(profile = NULL, seed = 1L, refresh = FALSE)`: applies the final Tab 3 population values to staged REF and CF synthpop snapshots and writes row-consistent trip totals and spread defaults for Tab 4. At this transition the final Tab 3 person/user counts are authoritative, but a trip-derived Tab 2 quota also remains authoritative. If the final people do not own enough observed trip rows, HUB assigns sampled donor trip patterns to those fixed users rather than silently reducing the trip total. No HM data are loaded.
- `build_results(profile = NULL, seed = 1L, refresh = FALSE)`: runs the current end-to-end calculation from the fully filled profile. It builds counterfactual data from the synthpop reference data, applies the HM death-share cycle lookup to ref/cf physical-activity exposure, and returns the updated `profile`, `reference_data`, `counterfactual_data`, compact `health_impacts`, and `results_data`. The full person-cycle table is released after these compact result objects have been built.
- `get_results_highlights()`: after `build_results()`, returns the three assessment-period totals for prevented deaths, saved life-years, and prevented disease cases.
- `get_results_options()` / `get_ui_options(option)`: return lightweight, configured IDs and labels for UI controls without loading appraisal data.
- `get_assessment_period()`: returns the canonical model/result horizon.

The intended Shiny flow is:

```
hub <- MIAMAHUB::Hub$new(cfg = hub_cfg)

setup_profile <- hub$get_appraisal_setup_inputs(mdata[["profile"]])

mdata[["profile"]] <- hub$build_reference_profile_defaults(mdata[["profile"]])
attr(mdata[["profile"]], "reference_defaults_report")

# After Tab 2 and before displaying Tab 3:
mdata[["profile"]] <- hub$build_refinement_profile_defaults(mdata[["profile"]])
attr(mdata[["profile"]], "refinement_defaults_report")

# After Tab 3 and before displaying Tab 4:
mdata[["profile"]] <- hub$build_trip_refinement_profile_defaults(mdata[["profile"]])
attr(mdata[["profile"]], "trip_refinement_defaults_report")

# After the UI has collected Tab 4 refinements:
result <- hub$build_results(mdata[["profile"]])
```

For inspection during development:

```
names(hub$get_profile())
str(hub$get_profile()$pop_total_ref_basic)
str(hub$get_profile()$pop_total_ref_advanced)
hub$get_request()$appraisal_input_values
hub$get_reference_ui_updates()
result$results_data$results_table
```

For a more realistic profile test that reads the actual MIAMA-UI `schemes/default.R`, fills a LAD selection, runs the reference-data pipeline, and creates an inspectable `profile_with_defaults` object, run:

```
source("inst/workflows/dev_profile_defaults_from_ui_default.R")
```

Developer helpers such as `load_reference_sources()`, `build_reference_data()`, `build_counterfactual_data()`, and `build_results_data()` remain available for workflow scripts and targeted testing. New MIAMA-UI integration should prefer the primary methods above, with `request` treated as HUB-internal state.

Configuration versus appraisal request

MIAMA-HUB now distinguishes between two different concepts:

- **config**: runtime and infrastructure choices
- **request**: appraisal-specific intent coming from `appraisal_inputs`

Examples of **config** concerns:

- file locations
- cache settings
- whether runtime uses the `sample`, `leeds`, or `full` data profile
- which parquet directories provide local synthpop sources

Examples of **request** concerns:

- geographic scope (`geo_level`, `geo_id`)
- result aggregation (`res_aggregation`)
- selected outcomes and modes
- later, counterfactual population and trip changes

A key recent change is that HM source selection is no longer driven by config. Instead, `results_request$res_aggregation` maps to the HM dataset suffix:

- `total` -> `overall`
- `timeline` -> `cycle`

This keeps appraisal logic in the request layer and runtime concerns in config.

2. Tab 1: Set up the appraisal

Geographic levels and options

MIAMA-HUB exposes lightweight geography helpers for UI select controls:

- `get_geo_levels(cfg)` returns available levels.
- `get_geo_options(cfg, geo_level)` returns selectable `geo_id` / `geo_name` rows for one level.
- `get_geo_details(cfg, geo_id, geo_level = NULL)` resolves one selected option to summary fields for Tab 1: `location`, `geographic_scale`, and `administrative_location_id`.
- `get_geo_name(cfg, geo_level, geo_id)` resolves one selected geography to its display name.
- `Hub$get_geo_options(geo_level)` exposes the same options through the session object.
- `Hub$get_geo_name()` resolves the current session's selected geography.

The current lookup is built from the full synthpop individual attributes parquet source and cached at `data/lookup/geo_options.rds`. The same full-derived lookup is also stored at `inst/extdata/data/lookup/geo_options.rds` for installed package deployments. If the cache is missing, the first call builds it from `cfg$sources$sp_attributes`; subsequent calls read the small RDS lookup instead of loading full reference data. The returned table includes:

- `n_individuals`: synthetic rows in the full 5% sample
- `person_weight`: currently 20
- `population_size_synth_scaled`: `n_individuals * person_weight`
- `population_source`: "Census 2021 5% synthpop scale"

The currently supported levels are:

- `eng`: one England-wide option
- `reg`: English regions, using `region`

- **lad**: local authority districts, using **lad25cd** and **lad25nm**

The alias **region** is accepted by **get_geo_options()** and normalized to **reg**. Grouped LAD and MSOA options are not derived yet because the current synthpop attributes source only exposes **region**, **lad25cd**, and **lad25nm**.

Tab 1 can call the lightweight geography helpers directly, without creating a Hub object:

```
lad_options <- MIAMAHUB::get_geo_options(mdata[["hub_cfg"]], "lad")
geo_details <- MIAMAHUB::get_geo_details(mdata[["hub_cfg"]], input$geo_id)

geo_details$location
geo_details$geographic_scale
geo_details$administrative_location_id
geo_details$population_size_synth_scaled
```

Timeframe value conversion

convert_timeframe_value() is the shared HUB/UI converter for values expressed per day, week, or year. It is exported and does not require a Hub object or loaded reference data:

```
annual_trips <- MIAMAHUB::convert_timeframe_value(
  old_timeframe = "week",
  old_value = 100,
  new_timeframe = "year",
  datatype = "trips"
)
```

The equivalent session method is **hub\$convert_timeframe_value(...)**. Both use the same implementation. For **datatype = "trips"**, the factors are **day = 1 / 7 week**, **week = 1**, and **year = 52.1775 weeks**. This datatype is also used for total distance and total duration, which accumulate linearly over time.

For **datatype = "users"**, the current conversion factor is 1 for every day/week/year combination. This is a documented provisional assumption because distinct people do not scale linearly with the observation period. The API lets us replace those factors later without changing UI call sites. HUB reference extraction and counterfactual normalization call the same function, preventing UI display conversions from diverging from calculation conversions.

Appraisal summary values

The Tab 1 summary should reuse canonical user-selected fields where they already exist, rather than creating duplicate summary-only parameter names. In particular, **geo_level** and **geo_id** remain the source of truth for the selected geography.

Summary-only fields should be reserved for values that are derived or displayed:

- **geo_name**: display name resolved from **geo_level** / **geo_id**
- **population_size**: display alias for the filtered reference population size
- **appraisal_name**: user-entered nickname for the appraisal

geo_name / **location** is available from the lightweight geography lookup and can be resolved before full reference data is built. Prefer **get_geo_details(cfg, geo_id)** for Tab 1 summary panels. **population_size** is the canonical summary value corresponding to both **pop_total_ref_basic** and **pop_total_ref_advanced**; it is available after reference data has been filtered and **extract_reference_ui_values()** has run. The R6 API still exposes a compact development helper:

```
hub$get_appraisal_summary_values()
```

which returns **geo_level**, **geo_id**, **geo_name**, **population_size**, and **appraisal_name**.

In **appraisal_inputs**, **input_source** is optional metadata used by HUB normalization: when omitted it defaults to **"user"**. For fields populated by HUB rather than typed by the user, set it explicitly to **"derived"** so UI code and developers can distinguish display values from user inputs.

3. Runtime configuration and source loading

Current development workflow for appraisal_inputs

Direct package-to-UI wiring is not finished yet. For development, MIAMA-HUB uses a small helper, `build_mock_appraisal_inputs()`, which creates a **minimal canonical mock** of the UI input object.

That mock is used in [inst/workflows/dev_workflow.R](#) to:

1. create a temporary appraisal input object
2. pass it through `receive_appraisal_inputs()`
3. inspect the flattened values in `request$appraisal_input_values`
4. use those values to build and test downstream modules such as `filter_reference_data()` and request-driven source loading

The intent is to expand this mock incrementally as new modules are implemented, rather than copying the full UI schema into MIAMA-HUB.

Configuration orientation

`miama_default_config()` is the main developer-facing index of configurable HUB behavior. Its top-level sections distinguish workflow controls (`workflow`, `arrow`), model assumptions (`population`, `physical_activity`, `spread`), stable presentation metadata (`results`), and infrastructure (`sources`, `cache`, `output`). Override individual leaf values rather than replacing the whole config wherever possible.

Tab 5 health outcome definitions live in `cfg$results$outcomes`. Each named definition records its UI label, outcome type, category, unit, benefit direction, reference HM source columns, and whether it is selected by default. Composite outcomes such as CVD and all cancers explicitly list all columns that are summed. Results preparation consumes this same catalogue, so UI choices and calculations do not maintain separate hard-coded mappings.

Request-driven source loading

`load_reference_sources()` now accepts both config and request sections.

The current loading strategy is:

1. use `reference_request` to prefilter synthetic population attributes by geography where possible
2. derive matching `census_id` values from the filtered attributes
3. filter synthpop trips by the same geography when possible; for unscoped or England-wide requests, use the matched `census_id` values to keep trips aligned with loaded attributes
4. load one-row-per-person HM `overall` outcomes only when results are built; cycle/death-share outcomes remain deferred to the counterfactual health step

This is intended to reduce unnecessary data transfer and memory use, especially once synthetic population data is available in parquet format.

Parquet reads run through a small Arrow runtime wrapper. By default `miama_default_config()` sets `cfg$arrow$cpu_count = 1L` during HUB source loading, restores the previous Arrow thread setting afterwards, and triggers garbage collection after the read block. This avoids large parallel decode buffers in memory-constrained Shiny/Connect sessions. Set `cfg$arrow$cpu_count = NULL` to leave Arrow's current thread setting unchanged.

4. Tabs 2-4: Specify and refine active travel

The three input and refinement tabs operate on one staged appraisal rather than independent datasets:

Tab	User-facing question	HUB transition
Tab 2	How much active travel is assessed in REF and CF?	Convert the selected users, trips, distance/duration, or mode-share route into staged REF and CF person/trip scopes.
Tab 3	Who is represented and who changes?	Recalculate population counts and distributions from the Tab 2 scopes, then apply population-level refinements and candidate weights.
Tab 4	Which trips change?	Measure row-consistent trip defaults from the final Tab 3 person/user scopes, then apply trip distance, induced-trip, and source-mode assumptions.

The profile is the handoff object at every transition. An applicable submitted `input_value` takes precedence over a HUB-generated `default_value`; inactive conditional fields must not be marked as submitted by the UI.

Editable assumptions: complete, do not replace, user inputs

The coordinated `codex/assumptions-contract` draft centralizes this behavior in `R/appraisal_assumptions.R`. Cards show the missing frequency, distance **or** duration, and speed needed for the selected input route. Already entered parameters and unchanged distributions are not duplicated. Defaults and edits are stored separately, with their source visible to the user.

The default hierarchy is assessed REF, geographic source, stored England rate, then fixed fallback; speed uses fixed configuration. Geography initialization normally has only source data available. Defaults then remain stable until explicitly rebuilt, avoiding feedback from CF sampling into its own assumptions. PT means walking access, not the full public-transport journey.

Saved overrides affect the corresponding volume conversion, candidate weighting or changed-trip active minutes on the next calculation. They do not rewrite observed REF activity or discard accepted person/trip totals. Explicit advanced distance inputs take precedence over a simple mean. Completed results retain `results_data$assumptions`, also used by table/report exports.

See [Assumptions that complete an appraisal](#) for the field catalogue, precedence rules, source limitations and review checklist. The UI implementation is one shared `utilites/assumptions_cards.R` controller on the matching UI branch; no publishing configuration changes are needed.

Reference UI value extraction

Tab 2 staging ignores user-count widgets when the active input unit is trips, distance/duration or mode share. It also ignores basic population-modal fields in the advanced workflow. Hidden generated zeroes must not override the active volume input. With no observed e-bikers, the default REF remains zero; positive CF e-bike volume can still recruit recipients and use cycling donor patterns. Explicit user counts on the users route and population edits in the basic workflow remain authoritative. This is covered by `test-ebike-tab2-handoff.R`.

Age/PA category payloads count the same accepted REF/CF mode-user flags as the Tab 3 table. They do not reclassify trip-derived users from unchanged individual activity columns. With all categories selected, totals therefore equal the unfiltered table, including CF e-bikers absent from the source population.

`extract_reference_ui_values()` derives compact status-quo values from filtered `reference_data`. The low-level function still returns a named `ui_updates` list for developer inspection and tests. The UI-facing R6 method is `Hub$build_reference_profile_defaults(profile)`: it runs the synthpop-only reference-default pipeline and writes all matching reference values into the profile's `default_value` fields. HM outcome data are not loaded for this step; health-model data are first needed when `build_results()` applies ref/cf health outcome calculations.

The extraction and profile-write responsibilities are intentionally separated:

- `reference_data_extract_reference_ui_values.R` derives all reference values that HUB currently knows how to calculate.
- `api_reference_profile_defaults.R` writes each derived value into `default_value`, into category metadata `additional_data`, or into both `default_value` and a declared `additional_data$default_value_backup`, when the matching profile field exists.

Intended UI usage:

```
hub <- MIAMAHUB::Hub$new(cfg = hub_cfg)
mdata[["profile"]] <- hub$build_reference_profile_defaults(mdata[["profile"]])
```

This is the Tab 1-to-Tab 2 operation and describes the unmodified geography source population. Before opening Tab 3, call the staged operation:

```
mdata[["profile"]] <- hub$build_refinement_profile_defaults(
  mdata[["profile"]],
  seed = 1L
)
```

This call does not reload parquet or HM data. It converts filled Tab 2 reference volumes into REF person/user/trip scope flags, applies filled Tab 2 counterfactual inputs to a synthpop-only CF copy, and derives the advanced population table and spread defaults from those row-level snapshots. The returned `refinement_defaults_report` records updated fields, reset hidden inputs, the scope and counterfactual reports, and the seed.

The Tab 3 `pop_new_current_perc` slider is initialized from the realized current/new-user split in that staged CF snapshot when additional mode users were produced. The calculation pools mode-user memberships across assessed modes and is recorded as `realized_new_user_percent` in the refinement report. If there is no positive user change, the configured/profile default is retained.

For user-count input, each mode-specific Tab 3 row is the corresponding Tab 2 count. For trip-count input, it is the number of unique people owning the selected active-mode trips. Unless the user supplied an explicit population total, HUB estimates the common assessed REF/CF population using the pooled selected-mode rate in the source population:

```
estimated_people = source_people *
  sum(requested_mode_users_or_trips) / sum(source_mode_users_or_trips)
```

This pools all selected modes rather than choosing the largest standalone mode estimate. A person contributing to two modes is represented in both the requested and source sums, and per-mode estimates remain available in `reference_scope_report$person$mode_estimates` for diagnosis. The estimate is bounded below by every explicit mode-user count. Larger requests can reuse eligible source donors rather than being capped at the source population size.

The Tab 3 table is initialized from the staged REF and CF snapshots. Both sides remain independently editable. Basic percentage and age/PA-category controls scale **both** scenarios from preserved staged backups, so they change the overall appraisal reach without erasing the Tab 2 REF/CF contrast. Category counts use `additional_data$ref` and `additional_data$cf`; each contains absolute row counts for its scenario. Repeated slider movement does not compound rounded values. A later manual table edit overrides the generated value until a refinement control is moved again. Advanced age, sex, and PA sliders leave counts alone and modify CF candidate sampling weights.

The same staged totals and mode-user counts are also written to the Tab 2 basic population fields (`pop_total_*_basic` and `pop_number_*_*_basic`). Therefore, when the optional basic population modal is opened after entering trips, distance/duration, or mode shares, its starting values should describe the Tab 2-implied appraisal population rather than reverting to the complete source geography. For non-user input routes, REF and CF total population remain equal; the mode-specific counts describe the different REF and CF travel snapshots.

The shared population total is operational as well as explanatory: it defines the assessed person rows, the eligible non-user pool, and population-based rate denominators. Mode-specific REF/CF rows determine active-mode user targets.

MIAMA-UI integration contract The UI should use the two profile-building methods at distinct transitions:

1. At the end of Tab 1, call `build_reference_profile_defaults()` once to load the selected geography and populate the baseline defaults used in Tab 2.
2. After saving the current Tab 2 controls and before rendering Tab 3, call `build_refinement_profile_defaults()` to build the staged REF and CF snapshots and replace the advanced population/spread defaults.
3. After saving Tab 3 and before rendering Tab 4, call `build_trip_refinement_profile_defaults()`. It applies the final Tab 3 population scopes and initializes Tab 4 from the carried or inferred REF/CF trip targets. Trip-based Tab 2 inputs remain fixed through this transition. Do not call `build_reference_profile_defaults()` again; that would overwrite the staged handoff with geography-wide defaults.

The Tab 4 `induced_trips_percent` slider is initialized from the realized share of added CF trips represented by induced rows. This may differ from the Tab 3 new-user percentage because people and trips are separate mechanisms. When no trip addition has yet established a realized split, HUB uses the effective Tab 3 `pop_new_current_perc` value as the Tab 4 starting assumption. The source and value are retained in `trip_refinement_defaults_report`.

HUB owns source-data loading, REF scoping, CF staging, population estimation, category metadata, and profile defaults. MIAMA-UI owns rendering controls, reactive updates, and persisting user edits into `input_value/is_filled`. The UI should not copy REF defaults into CF fields itself; HUB initializes the no-change CF defaults while preserving genuine submitted CF values.

Before this call, MIAMA-UI must copy the selected Tab 1 setup values into `mdata[["profile"]]`, especially `geo_level`, `geo_id` for non-England geographies, `ui_version`, `modes`, `intervention_type`, and `data_source`. The R6 method now fails fast if reference geography is missing, because an unscoped call can otherwise trigger a much larger parquet load than intended.

The returned profile keeps `input_value` and `is_filled` unchanged. Fields that do not exist in the UI profile are skipped and listed in the diagnostic `reference_defaults_report` attribute. This attribute is not part of the appraisal schema and does not drive UI or HUB behavior. It records `updated_fields`, `additional_data_fields`, `default_value_backup_fields`, `mirrored_cf_fields`, `mirrored_cf_sources`, `skipped_fields`, and their counts so developers can reconcile calculated HUB values with fields available in the UI profile. HUB writes broad defaults and leaves conditional display choices to MIAMA-UI.

Every canonical paired counterfactual control receives a no-change starting default from its reference control. HUB pairs existing `_ref/_cf` and `_ref/_cf` profile fields, covering user and trip counts, distance/duration, mode share, mode-specific spread sliders, and the `pop_total_* / pop_number_*` controls in both the basic and advanced UI. The copy affects `default_value`; a user-submitted counterfactual `input_value` and its `is_filled` state are preserved. Advanced population totals and per-mode counts additionally preserve the geography-derived no-change value in `additional_data$default_value_backup`. Tab 3 may subsequently change the live `default_value` while sliders or category refinements are active without losing the original value. Rebuilding reference defaults, such as after changing geography, deliberately refreshes both the default and its backup. Pairing is constrained by the actual profile schema, so a calculated reference field without a corresponding CF control is not invented dynamically.

Tab 3's `pop_target_age_groups` and `pop_target_pa_groups` are different: their categorical selections remain in `default_value`, while HUB replaces their `additional_data` with counts derived from the filtered synthetic population. Under both `additional_data$ref` and `additional_data$cf`, every configured category contains absolute `pop_tot`, `pop_walk`, `pop_bike`, `pop_ebike`, and `pop_pt` row counts. Age categories come from `age1year` and `cfg$population_refinement$age`; PA categories come first from `mmets`, then `mmet_wkhr`, or otherwise reconstructed weekly MMET-hours from walking, cycling, and sport activity using configured intensities. Mode counts use the corresponding person-level duration column when available, with trip evidence as the supported fallback. Unavailable mode evidence produces `NA` rather than zero and is recorded in the extraction notes. These values are synthetic-profile row counts; population scaling is applied later in results, not to this UI metadata. The category sets are exhaustive: under-18 and other/unknown age groups, and an unknown PA group, prevent records from silently falling outside all choices. The five-bin `cfg$spread` categories remain separate because the spread-bar redistribution functions require exactly five bins.

The writer is schema-driven. A profile control whose `additional_data` contains `default_value_backup` is treated as a normal default field with a backup; other controls declaring `additional_data` receive a same-named derived `ui_updates` metadata payload. HUB must still implement the corresponding extractor; it does not invent new category data merely because the slot exists.

HUB session state and refresh behavior

`Hub$new(cfg = hub_cfg)` creates a session-scoped object. Construction is light: it stores configuration only and does not load parquet data. Synthpop reference default data are first loaded when `build_reference_profile_defaults()` calls `build_reference_default_data()`. HM outcome data are loaded later by `build_results()` / `build_counterfactual_health_outcomes()`.

The object keeps intermediate reference objects only while they remain useful:

- `reference_default_data`: synthpop-only reference data for UI defaults and counterfactual construction
- `reference_default_ui_values`: compact extracted defaults for the active profile
- `refinement_reference_data`, `refinement_counterfactual_data`: temporary synthpop-only REF/CF snapshots used to pre-populate Tab 3 consistently after Tab 2
- `reference_sources`, `reference_data_raw`, `reference_data`: HM-enriched reference objects used by lower-level workflow scripts and result building

When `build_results()` follows reference-default extraction, it reuses the already geography-filtered synthpop rows and loads only matching HM outcomes. It does not read the attributes and trips parquet sources a second time. Once compact defaults and health-enriched reference data exist, redundant source and default row tables are released before the counterfactual and cycle objects are materialized. Trip data retain the SP person/trip fields needed by sampling but do not repeat wide HM outcome columns on every trip row.

When `build_reference_profile_defaults(profile)` receives a profile, HUB compares the new flattened input values with the previous request and invalidates cached state as needed:

- Changes to `geo_level`, `geo_id`, or `res_aggregation` clear reference data caches, so the next call reloads/rebuilds reference data.
- Lighter changes such as `modes`, `ui_version`, `denominators`, `units`, or `timeframes` keep loaded reference data but clear extracted default values, so defaults are recalculated from the same data.
- Counterfactual and results objects are cleared whenever submitted profile values change. Counterfactual and Tab 5-only changes retain the compact reference defaults; only inputs that affect reference extraction clear and recompute them.
- Staged refinement snapshots are cleared when submitted profile values change. They are lightweight precursors to the health pipeline, not a second authoritative results cache.

Use `refresh = TRUE` only for an explicit forced recomputation when the inputs have not changed, for example during debugging or after replacing source files on disk. Normal UI navigation from Tab 2 back to Tab 1 and selecting a different location should not require `refresh = TRUE`; the geography change should invalidate and reload automatically.

The extractor accepts the flattened submitted values from `receive_appraisal_inputs()` so it can honor selected denominators, units, and timeframes. The default UI schema should use `week` as the baseline timeframe; year/day conversions are lightweight display recalculations and should not force a full reference-data reload.

Broad default-writing rules:

- `build_reference_profile_defaults(profile)` calculates the available reference defaults broadly and writes every matching field in the profile.
- It does not filter default writes by `ui_version`, `at_data_unit`, selected `modes`, or `trips_refine_method`; those profile fields control what the UI displays, not what HUB is allowed to precompute.
- All supported active modes are calculated where the required reference columns exist. Unsupported or unavailable mode fields are returned as NA and listed in the extraction report.
- Mode share defaults include both scalar fields (`mode_share_ref_*`, `mode_share_total_*`) and the aggregate `mode_share_ref` list used by the UI pie-input schema.

This broad-default approach minimizes UI/HUB round trips after Tab 1. Dedicated small methods can still be added for cheap recalculations, such as changing a trip count from week to year or day without reloading parquet data.

The current extractor covers Tab 2 reference fields for:

- user counts (`users_count_ref_*`)
- trip counts (`trips_count_ref_*`)
- distance/duration amounts (`dist_dur_amount_ref_*`)
- mode shares and denominators (`mode_share_ref`, `mode_share_ref_*`, `mode_share_total_*`)

It also covers advanced Tab 3 and Tab 4 reference fields:

- basic and advanced population totals and per-mode population counts (`pop_total_ref_basic`, `pop_total_ref_advanced`, `pop_number_ref_*_basic`, and `pop_number_ref_*_advanced`) plus the canonical `population_size` summary value
- mode-specific population distribution anchors (`pop_spread_age_mean_ref_*`, `pop_spread_sex_prop_ref_*`, `pop_spread_pa_mean_ref_*`, `pop_spread_pa_sex_prop_ref_*`)
- mode-specific compact population spread bars (`pop_spread_bars_ref_*`)
- mode-specific compact PA spread bars (`pa_spread_bars_ref_*`, plus temporary `pop_spread_pa_bars_ref_*` aliases while the UI field names settle)
- trip totals and per-mode trip counts (`trips_number_total_ref`, `trips_number_ref_*`)
- mode-specific trip distribution anchors (`trips_spread_mean_ref_*`, `trips_spread_util_prop_ref_*`)
- mode-specific compact trip spread bars (`trips_spread_bars_ref_*`)
- target-specific source-mode diversion pies (`trips_diversion_sources_[walk|bike|ebike|pt]`)

Individual-only walking and cycling user counts can be extracted from `walktime_wkhr` and `cycletime_wkhr`. PT user activity is derived from the walking component of PT main-mode trips and therefore requires trip data. E-bike reference user and trip volumes are explicitly zero because the source does not distinguish conventional bicycles from e-bikes. Cycling observations remain cycling and are used only as donor distributions for e-bike counterfactuals, avoiding double counting. Trip counts, trip-level distance/duration values, and mode-share values require `reference_data$trips`. The returned `extraction_report` records skipped fields and notes when a requested UI value cannot be derived from the currently available columns. It also includes `spread_bar_values`, a flattened report table of every compact reference spread bar payload with its source profile field, category, plotted variable, percent, and proportion.

All four UI modes have an explicit runtime interpretation:

- walking uses the walking component of non-PT trips
- cycling uses observed bicycle records
- e-biking starts from zero reference volume when unobserved. Cycling proxies use provisional factors of 1.30 for distance, 1.20 for speed, and 1.30 / 1.20 for duration; user profiles and MMET intensity remain cycling proxies. See [E-bike proxy calibration](#).
- public transport uses PT main-mode records for travel volume, while only the walking-access component contributes physical activity and health exposure

Current threshold assumptions are deliberately simple:

- walking users are individuals with `walktime_wkhr > 0`
- cycling users are individuals with `cycletime_wkhr > 0`
- walking trips have positive walking time or distance and are not PT trips
- cycling trips have `trip_cycletime_min > 0` or `trip_cycledist_km > 0`
- walk-to-public-transport trips have recognizable public-transport `trip_mainmode` values plus positive walking time or distance

Future refinements should make these thresholds mode-specific and configurable. Likely examples include defining walking users as `walktime_wkhr > 2`, or using minimum trip-count thresholds such as more than 10 trips.

Spread plot payloads and slider redistribution

MIAMA-UI still owns modal presentation and plotting. HUB owns the data calculation. The intended split is:

- UI uses its plotting function, currently derived from the POC `plot_dist_bars()` implementation.
- HUB provides mode-specific reference slider anchors, such as `pop_spread_age_mean_ref_bike` and `trips_spread_mean_ref_walk`.
- HUB also provides compact reference bar values in profile `default_value` fields.
- UI can call `hub$get_spread_bar_values(...)` or `MIAMAHUB::spread_bar_values_from_slider(...)` to update counterfactual bars interactively when the user moves a slider. These cf bars are for display; UI only needs to submit the cf mean/proportion slider scalar fields.
- `Hub$build_results()` also attaches the same ref/cf spread payloads at `result$spread_data` and `result$results_data$plot_data$spreads`.

The compact spread bar object is a 10-row data frame: five categories crossed with two plotted variables. It contains `topic`, `scenario`, `category_order`, `category`, `category_midpoint`, `variable_order`, `variable`, `percent`, and `proportion`. This object is intentionally small and independent of raw individual/trip rows, so Shiny can update plots without keeping the full synthpop in memory.

Current reference spread topics:

- **pop**: five configured age categories crossed with `male` / `female`, based on selected-mode users. Defaults follow the POC labels: 18-29, 30-39, 40-49, 50-59, 60+.
- **trips**: five configured trip-distance categories crossed with `utilitarian` / `recreational`, based on selected active-mode trips. Defaults follow the POC labels: 0-2km, 2-5km, 5-10km, 10-30km, 30+km.
- **pa**: five configured physical-activity categories crossed with `male` / `female`, based on selected-mode users and available `mmets` / `mmet_wkhr` values; if needed, HUB reconstructs a pragmatic MMET-like value from walking, cycling, and sport hours. Defaults use the current POC category wording: `sedentary`, `low`, `moderate`, `high`, `very_high`.

Tab 3 population and PA spread lifecycle Tab 3 spread controls are mode-specific. Profile field suffixes follow the UI mode naming convention: `walking` -> `walk`, `cycling` -> `bike`, `ebiking` -> `ebike`, and public transport/walking-to-PT -> `pt`. All four paths are supported. E-bike reference bars use cycling as the donor distribution because the source has no separate e-bike observations.

At the end of Tab 1, UI calls:

```
mdata[["profile"]] <- mdata[["hub"]]$build_reference_profile_defaults(
  mdata[["profile"]])
```

That call loads the location-specific synthetic population once, derives all supported reference spreads, and writes them into matching profile `default_value` fields. The Tab 3 UI then reads the following fields for each mode suffix:

Topic	Reference bars	Reference mean slider	Reference proportion slider	Counterfactual mean input	Counterfactual proportion input
Age and gender	<code>pop_spread_bars_ref_{mode}</code>	<code>pop_spread_age_mean_ref_{mode}</code>	<code>pop_spread_prop_ref_{mode}</code>	<code>pop_spread_age_mean_cf_{mode}</code>	<code>pop_spread_prop_cf_{mode}</code>
PA and gender	<code>pa_spread_bars_ref_{mode}</code>	<code>pa_spread_pa_mean_ref_{mode}</code>	<code>pa_spread_sex_prop_ref_{mode}</code>	<code>pa_spread_pa_mean_cf_{mode}</code>	<code>pa_spread_sex_prop_cf_{mode}</code>

`pop_spread_pa_bars_ref_{mode}` is a temporary alias for `pa_spread_bars_ref_{mode}`. New UI code should use the shorter canonical `pa_spread_bars_ref_{mode}` name.

Reference population bars are calculated from current users of the selected mode. A user is identified from the positive mode-specific weekly activity column, or from linked mode-specific trips when that individual column is not available. The population spread uses one unweighted row per synthetic person. The PA spread uses the same mode-user filter and the individual's total weekly MMET exposure. If joined HM `mmets` are unavailable during reference-default extraction, HUB reconstructs the exposure from walking, cycling, and sport hours using the configured activity intensities.

Each reference bar payload is a joint distribution that totals 100 percent. The reference mean slider is derived from the five category totals and their configured midpoints, so it is a grouped-data approximation rather than the raw arithmetic mean. The reference proportion slider is the total share of the first variable (`male`).

The current profile distinguishes modal choices for distributions among current users versus new users, but HUB currently has one reference spread per mode and that spread describes current mode users. For a new-user modal, it is therefore the observed current-user pattern used as the starting assumption. The submitted counterfactual sliders constrain which non-users are sampled as new users. A separate empirical reference distribution of prospective new users is not currently available.

When `Hub$build_reference_profile_defaults(profile)` writes a mode-specific spread slider's observed `_ref_value`, it also writes that value to the paired `_cf_value` field's `default_value`. The counterfactual slider therefore starts at the no-change reference position. HUB does not set `input_value` or `is_filled`, and an existing user-submitted counterfactual value is preserved. This applies to age mean, sex proportion, PA mean, PA sex proportion, trip distance mean, and trip utilitarian proportion for each supported mode.

Slider redistribution logic `spread_bar_values_from_slider()` receives the compact reference bars and the two counterfactual slider values. Reference bar payloads also carry the reference mean and proportion as compact anchor columns. It does not load or retain raw synthetic population data.

1. HUB sums the 10 reference cells into a five-category marginal and a two-variable marginal.
2. The five-category marginal is exponentially tilted until its midpoint-based mean matches `cf_mean`. This preserves the reference shape as far as the requested mean allows and avoids generating one artificial average category.
3. `cf_prop` sets the first variable's share exactly; the second share is $1 - \text{cf_prop}$. Values may be supplied as 0-1 proportions or 0-100 percentages and are clamped to the valid range.
4. HUB combines the two counterfactual marginals with an outer product. The counterfactual joint bars therefore assume independence between age and sex, or between PA category and sex.
5. The function returns the same 10-row schema as the reference bars, with `scenario = "cf"`.

If one slider argument is `NULL`, its reference marginal is retained. If both are `NULL`, the exact reference joint cells are returned with only the scenario label changed. The same exact return now applies when the supplied CF slider values equal the reference anchors. Consequently the initial CF plot is an exact copy of the reference plot. The outer-product independence assumption is applied only after at least one slider moves away from its reference anchor.

Category labels, breaks, and slider midpoints live in `cfg$spread`, which is created by `miama_default_config()`. The current defaults are intentionally easy to override:

- `cfg$spread$age`
- `cfg$spread$trip_distance`
- `cfg$spread$pa`

The synthetic population stores single-year age (`age1year`), not a native age group. HUB therefore derives two related, explicitly ordered age schemes:

- `cfg$population_refinement$age` drives the Tab 3 category checkboxes, absolute category counts, appraisal-scope filters, Tab 5 health cube, and results filters. It is exhaustive: `Under 18`, the five adult bands from `18-29` through `60+`, and `Other/unknown`.
- `cfg$spread$age` drives only the compact age-and-gender spread bars and their mean-age slider. That chart retains five adult bands (`18-29` through `60+`) because the redistribution helper has a fixed five-bin contract. Under-18 and unknown records are excluded from that chart's denominator, not folded into another bar.

The checkbox filter and spread slider are separate refinement methods. HUB maps checkbox selections against the exhaustive population-refinement scheme and spread-slider weights against the five-bin spread scheme, so their labels may differ without changing category boundaries silently.

Other shared category contracts are:

- trip distance: `[0,2)`, `[2,5)`, `[5,10)`, `[10,30)`, and `30+ km`
- weekly PA: `(-Inf,0]`, `(0,10]`, `(10,25]`, `(25,50]`, and `(50,Inf]` MMET-hours, labelled `sedentary`, `low`, `moderate`, `high`, and `very_high`
- sex/gender: source `female = 0/1`, presented as `male / female`
- trip purpose: `utilitarian / recreational`; `mixed` is a UI input option that resolves to proportions of those two categories
- source trip modes: NTS codes are mapped through the constants in `R/constants.R`; walking, cycling, public transport, and car/private motor are the broad presentation groups, with unrecognised values retained as `other`
- setup-mode UI IDs are `walk`, `bike`, `ebike`, and `pt`; HUB normalizes these to `walking`, `cycling`, `ebiking`, and `pt` at the API boundary

The UI schema is intentionally static, so changes to configured IDs or labels must be mirrored in `MIAMA-UI/schemes/default.R`. Tests assert the current age boundaries to prevent Tab 3 and Tab 5 from drifting apart again.

The PA cutoffs are provisional weekly MMET-hour cutoffs: `(-Inf, 0]`, `(0, 10]`, `(10, 25]`, `(25, 50]`, and `(50, Inf]`. These should be reviewed against the intended PA exposure definition before production use.

The slider redistribution helper uses exponential tilting for the five-category numeric marginal. When the cf mean equals the reference mean, the category shape is unchanged; shifting the mean moves mass across

categories while preserving a smooth version of the reference shape. The second slider sets the first plotted variable's proportion: male for `pop/pa`, utilitarian for `trips`.

Concise MIAMA-UI implementation The recommended UI call is the exported stateless function. The R6 method `hub$get_spread_bar_values()` delegates to the same function and is equivalent, but no Hub state is needed for a slider-only recalculation.

Age/gender example for cycling (`mode_suffix = "bike"`):

```
pop_cf_bars <- shiny::reactive({
  ref_bars <- mdata[["profile"]][["pop_spread_bars_ref_bike"]][default_value]
  cf_mean <- input[["pop_spread_age_mean_cf_bike"]]
  cf_prop <- input[["pop_spread_sex_prop_cf_bike"]]
  if (is.null(cf_mean)) {
    cf_mean <- mdata[["profile"]][["pop_spread_age_mean_ref_bike"]][default_value]
  }
  if (is.null(cf_prop)) {
    cf_prop <- mdata[["profile"]][["pop_spread_sex_prop_ref_bike"]][default_value]
  }

  MIAMAHUB::spread_bar_values_from_slider(
    ref_bars = ref_bars,
    cf_mean = cf_mean,
    cf_prop = cf_prop,
    topic = "pop"
  )
})

output[["pop_cf_plot_bike"]] <- plotly::renderPlotly({
  plot_dist_bars(pop_cf_bars())
})
```

The reference anchors are included in `ref_bars`, so UI does not need to pass them separately. For manually constructed/legacy bar payloads, the optional `ref_mean` and `ref_prop` arguments provide the same no-change check.

PA/gender uses the same pattern with these substitutions:

```
ref_bars <- mdata[["profile"]][["pa_spread_bars_ref_bike"]][default_value]
cf_mean <- input[["pop_spread_pa_mean_cf_bike"]]
cf_prop <- input[["pop_spread_pa_sex_prop_cf_bike"]]

pa_cf_bars <- MIAMAHUB::spread_bar_values_from_slider(
  ref_bars = ref_bars,
  cf_mean = cf_mean,
  cf_prop = cf_prop,
  topic = "pa"
)
```

The exact plotting call depends on the UI-owned `plot_dist_bars()` signature. HUB returns long-format data with `category`, `variable`, `percent`, and ordering columns; UI should use `category_order` and `variable_order` rather than relying on alphabetical order.

Counterfactual bar data are display-only and are not written to a profile field. UI stores/submits only the mode-specific counterfactual mean and proportion fields by setting their `input_value` and `is_filled` values. During `Hub$build_results(profile)`, HUB calls the same redistribution function again, uses the reconstructed bars as sampling constraints, and returns the compact reference/counterfactual pairs at:

```
result$spread_data$by_mode$bike$pop$ref
result$spread_data$by_mode$bike$pop$cf
result$spread_data$by_mode$bike$pa$ref
result$spread_data$by_mode$bike$pa$cf
```

If the current UI schema does not yet include one of these ref bar fields, `build_reference_profile_defaults()` reports it in `reference_defaults_report$skipped_fields`. That is expected during migration: HUB can calculate the value before MIAMA-UI has a place to store/display it.

Applying these counterfactual spread percentages back to actual individual/trip rows is separate from plotting. The UI does not submit cf bar payloads. During counterfactual/result building, HUB reconstructs internal cf bars from the mode-specific ref bar defaults plus submitted cf mean/proportion scalar fields; the sampling code then treats those internal compact bar payloads as higher-priority constraints:

- `pop_spread_bars_cf_*` supplies the cf age-category marginal and male proportion for mode-specific individual sampling.
- `pa_spread_bars_cf_*` supplies the cf PA-category marginal for mode-specific individual sampling, while its male/female split is available for display and reporting.
- `trips_spread_bars_cf_*` supplies the cf distance-category marginal for mode-specific trip-shift sampling. Existing shifted trips are restricted to utilitarian candidates and induced rows are classified as recreational. The legacy utilitarian-proportion component is descriptive and is not an independent sampling constraint.
- If those compact payloads are absent, the older permissive hooks remain: `agecat_1_prop_cf ... agecat_5_prop_cf` and `distcat_1_prop_cf ... distcat_5_prop_cf`.

This is still a sampling approximation: candidate rows remain real rows, and the category proportions are used as sampling weights rather than as exact integer constraints. For large samples, weighted row sampling should usually move the selected-row distribution close to the target. Exact quota sampling would only become important if the UI/reporting contract requires the final sampled rows to match each five-category marginal exactly, or if small samples produce visibly unstable modal summaries.

Combined constraints can assign zero probability to most candidates in a small sample. If fewer positive-weight candidates remain than requested rows, HUB selects all positive-weight candidates first and samples only the unavoidable remainder uniformly from the zero-weight pool. This prevents `sample()` failures without discarding the constraints entirely. The affected change records `sampling_fallback` or `trip_sampling_fallback`, and the counterfactual report notes the number of relaxed rows.

Reference spread defaults also guard against sparse packaged samples. If a mode-specific subset has no usable observations inside the configured categories, HUB returns the filtered geography's overall population or trip spread instead of zero bars and an NA slider mean. The extraction report records this fallback. This keeps Tab 3/4 controls renderable while making clear that the displayed distribution is not mode-specific in that edge case.

The counterfactual report records which sampling constraints were active for a change in `sampling_constraints`, e.g. `sex`, `age`, `pa`, or `distance`.

Remaining clarifications:

- Final age, PA, and trip-distance category definitions can now be changed in `cfg$spread`, but the production defaults still need review.
- The PA modal now has a real PA-value basis in HUB, but the exact PA exposure definition and cutoffs still need review.
- Field names for the UI schema are mode-specific in both `MIAMA-UI/schemes/default.R` and `MIAMA-UI/schemes/appraisal_inputs.R`. Current suffixes are `_walk`, `_bike`, `_ebike`, and `_pt`. Aggregate unsuffixed fields still exist as backward-compatible selected-mode summaries, but mode-specific modals should read/write the suffixed fields.

`trips_spread_util_prop_ref` classifies trips as utilitarian unless `trip_purpose` looks recreational, leisure, sport, exercise, holiday, visit, or social.

For plausibility checks against the real MIAMA-UI profile object, use `inst/workflows/dev_profile_defaults_from_ui_def`. It loads `MIAMA-UI/schemes/default.R`, fills minimal Tab 1 setup inputs, runs `build_reference_profile_defaults()`, and shows fields populated in `default_value`. From a shell, run workflow scripts with `Rscript --vanilla` to avoid unrelated project startup/renv activation.

To test reference defaults against the full synthpop source without editing the workflow script:

```
MIAMA_DEV_DATASET_SIZE=full \
MIAMA_DEV_GEO_ID=E08000025 \
Rscript --vanilla inst/workflows/dev_profile_defaults_from_ui_default.R
```

`build_reference_profile_defaults()` uses a synthpop-only reference-default path, so full-data Tab 2/3/4 defaults do not require full HM outcomes. `pop_total_ref_basic` and `pop_total_ref_advanced` remain the filtered synthetic- population row count; `population_size` is overridden from the geography lookup's scaled population where available.

User-entered `pop_total_ref_*`, `pop_number_ref_*`, `users_count_ref_*`, and Tab 2 active-travel volume values are applied when results are built. Volume may be entered directly as trips or indirectly as distance, duration, or mode share. HUB first converts each supported indirect representation to a canonical weekly trip-row target, then applies the same reference-scoping machinery used for `trips_count_ref_*`. These inputs define an appraisal snapshot within the full geography rather than changing observed reference behaviour. When no explicit total exists, user/trip volumes imply an affected population through the pooled source-rate calculation described above. HUB retains the full geography as the source pool and adds explicit `ref_in_scope` / `cf_in_scope`, mode-specific user-scope, and mode-specific trip-scope flags. The resulting `reference_scope_report` records the derivation, requested and realized counts. Targets larger than the available strata currently fail explicitly; representing them requires agreed multiplicity weights rather than duplicated health trajectories.

For England-wide schema default review values, use `inst/workflows/dev_extract_england_schema_default_values.R`. It reads the full local synthpop parquet sources and performs filtering and aggregation lazily in Arrow. It writes two compact, package-bundled lookup files. This is an explicit developer regeneration step; app startup and appraisal sessions only read the resulting constants:

- `inst/extdata/data/lookup/england_mode_default_candidates.csv` contains the underlying England metrics by mode and measurement basis: trips per weekly user, trip distance, trip duration, speed, and weekly distance/duration per user.
- `inst/extdata/data/lookup/england_schema_default_candidates.csv` maps those metrics to current MIAMA-UI field names and labels each value as a review candidate, unavailable, or requiring a schema decision.

The canonical editable assumptions are `default_trips_per_user_per_week_*` and `default_trip_distance_*`. One weekly trip rate supports both conversions: `trips = users * rate` and `users = trips / rate`; HUB does not store a separate inverse rate. For a trip target, the implied weekly user count is retained in the counterfactual report; trip rows remain the operative sampling unit. Basic trip sampling uses `default_trip_distance_*` as its preferred mean while retaining the observed raw distance of each selected reference trip. Advanced Tab 4 mean distance inputs override that basic default. The Tab 2 user and trip modals intentionally edit the same weekly-rate profile field because they expose the same assumption and are not displayed simultaneously.

Installed-package code can resolve these files with `system.file("extdata", "data", "lookup", ..., package = "MIAMAHUB")`.

Walking and cycling use their active-trip component columns. Car and public transport use mutually exclusive numeric NTS main-mode codes and raw trip distance/duration. The available `MainMode_B04` field does not separate e-bike from bicycle, so independent England-derived e-bike candidates remain unavailable; runtime e-bike defaults instead use the documented cycling proxy. The current `distdur_default_*` field is also ambiguous because it is used for either distance or duration; the output reports both candidates with `needs_schema_split` rather than selecting one silently. Review these CSVs after regeneration before promoting values into MIAMA-UI schema defaults.

The R6 Hub wrapper still exposes developer helpers `build_reference_ui_values()` / `get_reference_ui_values()` / `get_reference_ui_updates()` for inspection. New MIAMA-UI integration should use `build_reference_profile_default` as the single call for populating reference fields. `get_population_size()` and `get_appraisal_summary_values()` are convenience helpers for development and summary displays.

5. Build the counterfactual and health effects

Reference appraisal scope

Before initializing CF, `apply_reference_appraisal_scope()` interprets submitted REF counts as system boundaries. Selecting a smaller population, fewer current users, or fewer active trips does not alter any person's travel, MMET value, or health trajectory. It only determines which observed rows count in the assessed REF snapshot. When no REF field was edited, all flags reproduce the previous full-geography behavior.

A zero REF activity value is valid and means that the assessed reference snapshot contains no users or trips of that mode. It must not imply that the appraisal contains no people when CF activity is positive. In that edge case, HUB applies the same source-rate population estimate to the CF user or converted weekly-trip target. If the source has no observable rate for a positive CF mode (notably e-bike), it retains the full geographic person pool as the defensible fallback. Direct trips, distance, duration, and mode-share inputs all pass through this

rule. A positive explicit population remains authoritative; an explicit zero is overridden only when needed to support positive CF activity.

CF starts from these flags. Added users are taken first from eligible baseline non-users already inside REF scope. If that pool is insufficient, HUB recruits eligible baseline non-users from the retained geographic source population and marks them `cf_in_scope`. This expands the assessed CF boundary without duplicating synthetic people or changing REF. Existing trips become eligible for switching only for people in CF scope. Cycle health data are calculated for the final CF scope, comparing each included person's original exposure with their CF exposure. `build_results()` returns the scope diagnostics as `reference_scope_report` as well as attaching them to `reference_data`.

Initialize counterfactual data

`init_counterfactual_data()` starts Step 6 by returning a 1:1 copy of filtered `reference_data`. The first UI-driven implementation is `apply_counterfactual_ui_values()`, which applies supported `_cf_` inputs to that copy and adds a compact `counterfactual_report`.

Feasibility rules and sampling sources

It helps to distinguish four populations that otherwise look like one number in the UI:

1. **Geographic source population.** This is every synthetic person loaded for the selected geography (for example, all 5,000 rows in the packaged Leeds profile). HUB keeps these rows available as evidence and possible donors.
2. **Scaled REF population.** Tab 2 describes the amount of existing active travel being assessed. HUB translates the entered users/trips/distance or duration into a population boundary and samples appraisal-person records. When source capacity is insufficient, eligible people and their trips are copied with new appraisal IDs and retained source-donor IDs.
3. **Refined REF population.** Tab 3 may narrow the scaled REF boundary. A percentage changes its size. Age and PA checkboxes are hard eligibility rules: an unchecked category cannot be sampled. HUB reconstructs the table totals from the category counts stored during Tab 2 -> Tab 3 staging, then draws a row-consistent snapshot from eligible source people. Advanced age/sex/PA spread controls instead change sampling probabilities; they are not hard quotas.
4. **CF population.** CF starts as a copy of refined REF. HUB then changes user and trip status. It can recruit baseline non-users from outside REF, but still using geographic source donors, with replacement when necessary.

Population replacement and scaling: source capacity is no longer a hard person-count ceiling. `R/population_donor_replication.R` supplies the shared fallback for basic/advanced staging and final reference construction. See the [capacity and scaling review](#). **Appraisal units:** one selected or copied person record represents one real appraisal person. Entering 1,000 users requests 1,000 user records, not six weighted records. Health aggregation defaults to `cfg$population$person_weight = 1`. Source expansion factors are provenance only, stored as `source_person_weight`; they neither reduce sampling targets nor multiply appraisal results. Donor reuse does not increase the number of independent source observations.

HUB constructs the snapshots in this order:

1. Read the submitted REF total, mode-user totals, and active-trip totals. If REF activity implies zero people while CF is positive, derive the person boundary from CF volume instead.
2. Identify source people allowed by the selected Tab 3 categories.
3. Select eligible REF people while meeting requested mode-user margins. When distinct donors are insufficient, reuse eligible donors with unique appraisal person IDs and reassigned trip IDs; preserve the link to each source donor.
4. Flag the selected people's observed trips as REF trips, then select the requested active-mode trip rows. Their original behavior and health exposure remain unchanged.
5. Copy the REF flags to initialize CF.
6. Apply CF user changes. In-scope baseline non-users are used first; additional baseline non-users are recruited from the geographic source pool when needed.
7. Apply CF trip changes. Existing eligible trips are switched where possible; configured induced trips are represented as additional trip rows.

8. Recalculate active-travel exposure/MMETs for changed people and compare their CF health trajectories with their original trajectories.

The operational rules are:

1. **REF uses observed donor behavior.** REF inputs select people and trip patterns from the filtered geography without changing donor behavior. A REF mode-user count cannot exceed REF total population, but eligible person and trip donors may be reused to supply larger appraisals. With no native e-bike users, a labelled cycling proxy can supply reference e-bike donors.
2. **CF starts from REF.** Every REF person initially belongs to CF and keeps their original behavior until selected for a change.
3. **More CF users may expand CF.** Eligible baseline non-users already in CF scope are converted first. Any shortfall is sampled from baseline non-users elsewhere in the retained donor pool, including eligible copies; those people and their existing trips are then marked `cf_in_scope`.
4. **Fewer CF users do not remove residents.** Selected current users become ex-users, but remain in CF scope with their other behavior intact.
5. **More CF trips use shifts plus induction.** Existing eligible utilitarian trips in CF scope are switched first according to the configured mechanism; the induced share and any shift shortfall are represented by added trip rows.
6. **Fewer CF trips switch trips away.** Eligible active-mode rows are changed to a configured alternative mode. A locked trip cannot be switched twice.

Consequently, a small REF population is not itself an error when CF is larger. An input can still be infeasible when there is no eligible donor evidence, when mode-user counts exceed the assessed total, or when requested joint mode margins require overlaps not represented by the donor profiles. These failures use class `miama_appraisal_input_error` and include the relevant profile fields, requested/available counts, and a user-facing correction hint.

What happens when a requested sample cannot be drawn? HUB does not use one generic fallback because different shortages mean different things:

- **Stale REF total after category selection:** HUB reconstructs REF and CF population-table values directly from the selected categories and the scenario-specific absolute counts stored in profile `additional_data`. This avoids relying on the timing of a Shiny `updateNumericInput()` message. If those stored counts are unavailable or still inconsistent, the category selection remains a hard eligibility rule: HUB reuses eligible donors to preserve accepted targets, without admitting excluded categories. Copies, original donor counts and proxy use are recorded in `population_replication_report`. A pool with no eligible evidence or incompatible population margins still produces a meaningful input error.
- **Too few positive sampling weights:** category membership is still eligible, but preferred age/sex/PA weighting cannot fill the sample. HUB takes all positive-weight candidates and samples the unavoidable remainder uniformly. This relaxation is recorded in the counterfactual report.
- **Too few CF non-users in scaled REF:** this is not an error. HUB recruits the shortfall from eligible baseline non-users elsewhere in the geographic source population and expands `cf_in_scope`.
- **Zero REF with positive CF activity:** this is not an empty appraisal. HUB infers a non-zero person boundary from the CF user or trip-equivalent volume; if no source rate exists, it uses the full geographic person pool. REF mode users/trips remain zero, while realized CF trip owners are marked as CF users.
- **Too few eligible people in the full geography:** HUB samples donor profiles with replacement, with unique appraisal IDs, copied trip ownership and retained donor IDs for health histories. `population_replication_report` records reuse. Copies increase modelled appraisal size, not the independent evidence sample.
- **Too few observed REF trips among the selected users:** HUB keeps the fixed trip target, samples observed geographic donor trip patterns with replacement, and assigns them to the fixed users. The warning and scope report expose how many modeled rows were required.
- **Too few switchable CF trips:** HUB uses the available mode-shift trips and represents the shortfall as induced trips. The realized mechanism counts are retained in the counterfactual report.

For example, the message “requested REF population 3,475; 2,939 satisfy the selected categories” means that 3,475 remained in the submitted population table while the age/PA checkboxes described a smaller eligible group. HUB now first replaces 3,475 with the selected-category total stored in the profile. If the accepted target still requires 3,475 records, HUB can copy from the 2,939 eligible donors. Copies are distinct appraisal records but not distinct observations.

Current implementation nuance: category totals are measured from the staged Tab 2 snapshot, but the final refined snapshot is reconstructed from eligible source rows using those totals and the same seed. It is therefore row-consistent and reproducible, but is not guaranteed to be a literal nested subset of the first intermediate random draw. Preserving exact row identity across that handoff remains a possible refinement if simulation testing shows a material effect.

Users: derive counterfactual number of active mode users

The current implementation supports `users_count_cf_*` in the basic UI, `pop_number_cf_*_basic` as the basic population-modal alternative, and `pop_number_cf_*_advanced` in the advanced UI for all four modes. HUB uses `ui_version` to select the applicable field family. These fields adjust the number of individuals with positive mode-specific weekly activity. The source population remains fixed, while assessed CF scope may grow when people outside scaled REF are recruited:

- if the counterfactual target is larger than the current reference count, eligible baseline non-users are sampled as `new_users`; in-scope candidates are used first and any shortfall is recruited from the retained geography
- if the target is smaller, existing users are sampled as `ex_users`
- new users receive mode activity values sampled from observed current users
- ex-users receive configured near-zero defaults, currently 0
- returned individual data includes explicit `user_walk`, `user_bike`, `user_ebike`, and `user_pt` indicators plus `cf_user_change`
- `mmets` is recalculated when present using HM constants: changes in `walktime_wkhr`, `cycletime_wkhr`, and `sport_wkhr` are converted to MMET deltas using factors 2.5, 5.8, and 7 respectively. These deltas are added to each individual's HM reference MMET value; unchanged individuals retain their reference exposure.
- if trip-level data is present, ex-users' active trips are shifted away from the active mode; new users trigger sampling of plausible non-active trips for mode shift where matching trip rows exist
- new-user trip shifts use the canonical mode-specific `default_trips_per_user_per_week_*` rate supplied by the appraisal profile. HUB calculates `new trips = new users * weekly trips per user`. If that assumption is absent, observed current-user trip counts remain the fallback.

Targets must be finite, non-negative, rounded integer counts. A target is feasible when the current CF users plus eligible baseline non-users in the full filtered geography can supply it. E-bike additions sample cycling donor profiles without moving the observed cycling reference count. PT additions retain PT as the travel mode but add only configured or donor-derived access-walking exposure to MMETs.

An explicit user-count input is authoritative: its CF-minus-REF difference already determines how many user-status changes are required. The separate current/new-user percentage is intended only for pathways where user counts are inferred from trips, distance, duration, or mode share. For those pathways HUB calculates a trip-equivalent changed population as `ceiling(abs(CF trips - current trips) / weekly trips per user)`. Existing users of that mode absorb the current-user share of additional travel; only `pop_new_current_perc` of the trip-equivalent increase is sampled as additional users. The profile defaults to `cfg$counterfactual$population$new_user_percent_default` (10%). The resulting mode-user scope populates the Tab 3 CF table without modifying individual activity columns, because changed trip minutes already provide the health exposure and changing both would double count it. E-bike is the explicit edge case: because the source has no observed e-bike users, current active-travel users provide its proxy current-user recipient pool.

For an explicit CF user target above REF, HUB retains the current mode users and recruits the required additional target-mode users from eligible baseline non-users across the retained geographic source, including people outside the scaled REF scope. The resulting current/new-user counts and percentages are recorded in `counterfactual_report$changes[[...]]$current_new_user_split`. The explicit count takes precedence over `pop_new_current_perc`; when it implies a higher new-user share, HUB records a non-fatal report note. The request fails if no eligible donor evidence exists. A shortage of distinct donors alone is handled by person replacement during the shared reference-construction step.

Trips: derive counterfactual number of active mode trips

The trip-count handler supports `trips_count_cf_*` and `trips_number_cf_*` for active-mode trip rows. It converts Tab 2 targets from total or mean-per-person values into a base-week trip count. Increases are split

into two mechanisms:

- **mode_shift**: existing non-active, utilitarian trips are switched to the active mode. Raw trip distance is preserved, and the active-mode distance/duration columns are populated from raw distance/duration.
- **induced_recreational_active**: by default 10% of additional active trips are treated as newly induced discretionary trips and added as new trip rows with recreational purpose.

The explicit mechanism parameter is **induced_trips_percent** in the submitted profile, with optional mode-specific fields such as **induced_trips_percent_walk**. When neither is supplied, HUB uses `cfg$counterfactual$trips$induced_trips_percent_default`; shifted trips are the complement. This is separate from both trip purpose and the percentage of activity assigned to new users, even when defaults happen to use the same number. Purpose fields no longer override the induced-trip parameter.

For increases, Tab 4 specifies one source-mode distribution for each assessed target mode using **trips_diversion_sources_**[w]. For example, **trips_diversion_sources_bike** answers: among existing trips shifted to cycling, what percentage previously used car, walking, e-bike, public transport, or another mode? Cycling itself is excluded from that pie. These percentages apply only to the mode-shift mechanism; induced trips are controlled separately by **induced_trips_percent**.

HUB combines the requested source shares with distance/spread weights when it samples eligible utilitarian donor trips. It renormalizes across source modes that actually have eligible candidates, so finite samples and unavailable donor pools can make realized shares differ from requested shares. Each change report records **diversion_source_field**, requested **source_mode_shares**, their source, and **realized_source_mode_shares**. Diversion pies are count-based conditional distributions and therefore require no total-trip, distance, or duration denominator.

The basic Tab 2 path does not require a diversion modal. If no pie was submitted, HUB first checks `cfg$counterfactual$trips$source_mode_shares` for the target mode. If no configured split exists, it derives a neutral default from the mode composition of eligible utilitarian trips outside the assessed active modes in the reference scope. This prevents one basic-mode target from consuming rows needed by another. Thus the default reflects donor availability rather than claiming an empirically observed intervention diversion rate.

Source-mode assumptions are separate from donor-profile assumptions. Because the current data cannot identify observed e-bike trips, cycling supplies the e-bike activity and trip-characteristic donor profile. It does not imply that all e-bike trips came from cycling. In the absence of a UI override, `cfg$counterfactual$trips$source_mode_shares$ebiking` assigns shifted e-bike trips equally across cycling, PT, and car source pools. The induced-trip share is excluded from this split. A submitted **trips_diversion_sources_ebike** pie replaces that configured distribution.

Decreases do not delete utilitarian travel demand. Instead, sampled active trips are shifted away from the active mode using the configured default destination, currently **car**. This reverse-direction assumption is deliberately separate from the source distributions used for increases.

Returned trip data includes explicit **trip_activemode**, **trip_utilitarian**, **cf_trip_change**, **cf_mode_shift**, **cf_induced**, and **cf_trip_locked** indicators. A row is locked as soon as it is shifted, shifted away, or induced; later mode handlers cannot select it again. Candidates for an active-mode increase also exclude trips already active in another assessed active mode. Together these rules prevent mode targets from cannibalizing one another or becoming dependent on their order in the UI profile. The change report records actual **mode_shift_n** and **induced_n**, and `counterfactual_report$comparison$changed_trip_rows` lists switched or induced trip rows.

Active-trip changes feed the physical-activity exposure used by the health model. For independent trip-count targets, HUB compares reference and counterfactual active minutes by (**census_id**, **nts_tripid**), converts the weekly minute difference to hours, and applies the configured MMET intensity (walking 2.5; cycling 5.8). Induced trips contribute their full active duration. Trip changes made only to mirror a new-user or ex-user status are excluded from this trip component because changed individual weekly activity already represents that exposure. The individual table exposes **cf_user_mmet_delta**, **cf_trip_mmet_delta**, and **cf_mmet_delta**, together with the attribution components **cf_mmet_delta_walking**, **cf_mmet_delta_cycling**, and **cf_mmet_delta_other_activity**. The report summarizes these components under `counterfactual_report$mmet_exposure`.

Advanced Tab 4 fields such as **trips_dist_value**, **trips_spread_mean_cf**, **induced_trips_percent**, and the diversion percentage fields are parsed and recorded. When compact cf spread bars are available, distance-based candidate selection uses the configured five-category distance distribution. Older direct category controls can

still plug into the `agecat_1_prop_cf ... agecat_5_prop_cf` and `distcat_1_prop_cf ... distcat_5_prop_cf` hooks, or the corresponding `_perc_cf` fields.

The intended simplified contract does not expose purpose as a second mechanism control: shifted existing trips are utilitarian and induced trips are normally recreational. Legacy scalar and spread-purpose fields should be retired in coordination with MIAMA-UI. The requested-versus-realized audit should report the purpose distribution implied by the mechanism.

Canonical conversion of alternative Tab 2 volume inputs

Tab 2 presents trip count, distance/duration, and mode share as alternative ways to specify active-travel volume. HUB normalizes the selected route before reference scoping or counterfactual sampling. The resulting internal target is always a weekly number of walking, cycling, e-bike, or PT trip rows:

1. Distance is converted to kilometres and duration to minutes.
2. Day or year amounts are converted to a reference week using `convert_timeframe_value()`.
3. An average-per-person amount is multiplied by the relevant REF or CF person scope; a total amount is used directly.
4. HUB measures the positive reference mean amount per active-mode trip and calculates `target trip rows = round(weekly total / reference mean per trip)`.
5. For mode share, HUB first calculates `mode amount = denominator total * mode percent / 100`. A trip denominator is already a trip-row target; distance and duration denominators use the same reference-mean conversion in step 4.

The conversion is applied independently to REF and CF. Derived values are written only into the internal flattened request as `trips_count_ref_*` or `trips_count_cf_*`; they do not overwrite the user's profile fields. The existing trip handler then scopes reference rows or samples counterfactual rows, including distance-distribution weights, diversion weights, trip locking, and the shifted/induced split. Consequently, mode share determines the per-mode sampling quota, while distance/duration determines a quota from the requested aggregate amount. The selected row characteristics determine the realized aggregate distance or duration and downstream MMET exposure.

In the advanced workflow, the values last displayed or entered in the Tab 4 trip table are the final trip-count contract. Its generated starting values already carry forward or derive from Tab 2, so accepting them preserves the upstream scenario; explicitly editing them supersedes it. Tab 4 diversion pies remain applicable because they control which eligible source trips are sampled, not the requested trip volume.

For Tab 2 distance, duration, and mode-share routes, the original aggregate is the provenance for the generated per-mode Tab 4 starting counts. Once a user explicitly changes a Tab 4 trip count, that count supersedes the generated target. HUB does not back-solve population or force the original kilometres, minutes, or shares afterward; final aggregate values and remaining-mode shares are measured from the resulting trip sample.

Cross-mode targets configured to use another active mode as a source are processed first; explicitly targeted donor modes are reconciled afterward. Only positive active-mode shares supplied by the UI or config permit this cannibalization. Locally inferred donor composition cannot consume active-mode rows. Every shifted or induced row is immediately marked `cf_trip_locked`, so the same physical trip cannot be selected by a later mode operation. Trip-pattern donors and exposure recipients are separate concepts: eligible trip rows may be sampled across the assessed population, then assigned to the selected current/new recipient pool. Original trip keys are retained for MMET differencing even when ownership changes.

The Tab 3 to Tab 4 handoff continues from the separate REF and CF snapshots already produced from Tab 2. If Tab 3 is accepted unchanged, HUB measures Tab 4 trip counts and distributions directly from those snapshots. If Tab 3 changes the assessed population, HUB re-scopes each snapshot independently while retaining trip-derived Tab 2 targets. If too few observed rows belong to the final users, geographic donor patterns are sampled with replacement and assigned to those users. `reference_scope_report$trips` records `donor_rows_added` and `allocation_method`. This preserves the one-way hierarchy: changing people in Tab 3 changes trips per user, not an authoritative upstream trip total.

The complete conversion assumptions are retained for inspection under `reference_scope_report$tab2_input_conversion` and `counterfactual_report$tab2_input_conversion`. Each mode entry identifies the source and denominator fields, source type and timeframe, canonical unit, reference mean per trip, optional population denominator, weekly total, and derived trip-row target.

Current limitations are explicit:

- **average_per_trip** distance/duration describes trip characteristics but does not identify total travel volume. HUB rejects it as a Tab 2 volume target; use a total or average per person, or use Tab 4 to refine trip distance.
- Mode-share distance and duration totals are currently interpreted as canonical reference-week totals in km/week and minutes/week. The UI schema should expose or state those unit/timeframe assumptions if alternatives are required.
- Conversion creates an integer row target. It does not calibrate selected rows to make realized aggregate kilometres or minutes exactly equal to the input. Finite candidate pools and advanced sampling constraints can therefore cause differences that must be reviewed in the requested-versus-realized report.
- E-bike results depend on the configured cycling proxy because no independent e-bike reference observations are available. PT health effects represent access walking only, not in-vehicle PT time.

Because the UI routes are alternatives, the selected **at_data_unit** controls which family is converted. Advanced Tab 4 trip targets retain their established precedence over Tab 2-derived targets.

Trip counts have one explicit contract: one synthetic trip row is one trip record. UI reference values, counterfactual targets, sampling, result tables, and plots all use weekly row counts. **weight_tripXhh** remains source metadata but does not change these targets. Shifted rows retain their source metadata and induced rows retain sampled donor metadata. Health-result population expansion is configured separately from this trip-count contract.

Parameter naming follows the same distinction used elsewhere in the package: ***_ref** values are measured from filtered reference data, while ***_default** values come from configured constants. **miama_default_config()** records the weekly activity timeframe, MMET intensities, and synthetic-population person weight; **miama_counterfactual_defaults()** exposes the counterfactual subset.

Sampling is centralized in **R/counterfactual_data_sampling_functions.R**. Sampling now means selecting existing candidate rows “as is”; the previous average-row and synthetic-distribution options were removed because they conflicted with the fixed-population mechanism. Candidate individual selection can be weighted by sex and five age-category targets. Candidate trip selection can be weighted by active-mode distance categories derived from local reference quintiles until final category breaks are available.

The R6 Hub wrapper exposes this step via **build_counterfactual_data()** and **get_counterfactual_data()**.

R/counterfactual_data_apply_ui_values.R is structured as a handler registry: **apply_counterfactual_ui_values()** builds shared context, then each handler owns one conceptual family of UI fields. Implemented handlers cover active-mode user targets and active-mode trip targets. The shared context first converts Tab 2 distance/duration and mode-share inputs into trip targets, allowing them to use the trip handler together with Tab 4 distance and diversion constraints. Future handlers can extend other trip-characteristic constraints without duplicating volume conversion. Keep each handler’s assumptions visible near the handler code, and keep cross-cutting validation, sampling, constants, and report helpers in their dedicated outline sections.

For validation, every counterfactual run adds **counterfactual_report\$comparison**, with side-by-side reference vs. counterfactual summaries for key active-travel and physical-activity columns. The current comparison includes individual-level sums, means, active-row counts, trip-level sums/means, trip row counts, and a **changed_ind_rows** table showing affected **census_id** values with **_ref**, **_cf**, and **_delta** columns. It also includes **changed_trip_rows** for trip mode switches and induced trips.

Counterfactual health outcomes

apply_counterfactual_health_outcomes() is the Step 7 bridge from changed counterfactual physical activity to health model outcomes. It uses the updated MIAMA-HM death-share cycle artifacts:

- **sp_cycle_outcomes_death_share**
- **mmet_d_cycle_lookup_death_share**

Those artifacts are produced by **MIAMA-HM/scripts/sp_hm_join.R**. HUB does not source or duplicate that script; it reads the processed artifacts and reproduces the scenario lookup-application logic documented in **MIAMA-HM/scenario_verification/scen_30to45_2mmets.qmd**.

Terminology in this step is explicit:

- **ref**: without-scheme/reference scenario
- **cf**: with-scheme/counterfactual scenario
- **delta**: **cf - ref**

- **baseline**: avoided here except when referring to HM source tables, because it can also mean the first simulation year

The function keeps all filtered individuals and all HM cycle rows. For the current `scheme_effect_duration = "longterm"` setting, the individual-level MMET delta from Step 6 is applied to every model cycle:

```
mmets_delta = mmets_cf_ind - mmets_ref
mmets_new   = mmets_cycle + mmets_delta
```

It then caps MMETs to the lookup maximum, overlaps the changed MMET interval with lookup bands, multiplies overlap width by per-MMET outcome slopes, and adds `d_*` outcome columns. Counterfactual values are derived as `ref + delta`; redundant `*_cf` columns are not materialized by default.

After applying the lookup, HUB also calculates health-adjusted life years (HALYs). This is distinct from the existing healthy-life-year (HLY) stream. Following the MIAMA-HM scenario-verification method, HUB reconstructs annual disease prevalence for each person. For chronic diseases:

```
prevalence[t] = prevalence[t-1] + incidence[t]
               - deaths[t] * death_share[t]
```

For depression, the remission/exits stream replaces the death-share term. Reference and counterfactual prevalence are calculated separately and divided by the corresponding population alive in each cycle. HUB then applies the packaged age/sex residual-disability (`pyld_rate`) table and age/sex/disease comorbidity-adjusted disability weights (`dw_adj`):

```
HALY = LY * (1 - pyld_rate) * (1 - sum(prevalence * dw_adj))
```

When no population remains alive in a person-cycle, prevalence disability and HALYs are set to zero. Person-cycles above the maximum age represented in the HALY parameter tables are retained in the health data but have NA HALY fields, so they are excluded from HALY summaries without discarding other outcomes.

The resulting person-cycle fields are `haly` (reference), `d_haly` (`cf - ref`), and, when `include_cf_columns = TRUE`, `haly_cf`. The two small parameter tables are packaged under `inst/extdata/data/health_data/haly_parameters`; their paths and the included disease streams are explicit in `cfg$results$haly`. The counterfactual health report records whether HALYs were available, the method, diseases used, and the unscaled total delta.

The death-share MMET lookup is much larger in memory than its parquet file. HUB derives the required (`age1year`, `female`, `mr_decile`, `cycle`) scope from the filtered people and cycle outcomes, applies Arrow filters before collection, and removes residual Cartesian combinations in R. Sample-mode calculations therefore do not materialize the complete England lookup table.

The low-level function returns `counterfactual_data` with:

- `health_outcomes`: full cycle-level reference and delta outcome table
- `counterfactual_health_report`: counts, MMET-delta summary, terminology, and summed outcome deltas
- `counterfactual_health_report$impact_overview`: compact outcome-level totals with `ref_total`, `cf_total`, `delta_total`, and `delta_per_1000_people`

Pass `include_cf_columns = TRUE` only for targeted model debugging that requires explicit `*_cf` columns. The high-level `Hub$build_results()` method consumes and releases `health_outcomes` after constructing compact `health_impacts` and `results_data`; plots and exports do not require the raw cycle table afterward.

Future work: add `scheme_effect_duration = "shortterm"`.

6. Tab 5: Present and export results

Appraisal summary population

The UI currently builds its summary in `modules/tab5/tab5Server.R`, using profile metadata and `hub$get_results_options()` for the assessment period. The Hub method now also returns `res_population_cf_total`, `res_population_cf_walk`, `res_population_cf_bike`, `res_population_cf_ebike` and `res_population_cf_pt`. `hub$get_appraisal_population_values()` returns just these five fields. The standalone `get_results_options(cfg)` remains a static configuration helper.

Counts come from the final assessed CF individual scope, not the geographic source population, REF, requested targets or only people with health outcomes. Each appraisal record counts once; mode-user groups can overlap and must not be summed to obtain the total. Missing CF data yields NA; an empty assessed CF scope yields zero. No cycling proxy is substituted for actual zero e-bike users.

`build_results()` fills matching `default_value` entries in its returned `profile` and freezes the values in `results_data$appraisal_summary`. Fields are output-only, not sampling targets. The corresponding fields are defined in UI `schemes/default.R` on UI dev; your UI profile copy needs the returned profile to see re-freshed defaults. Alternatively, read the Hub options directly:

```
options <- mdata[["hub"]]$get_results_options()
options$res_population_cf_total
options$res_population_cf_bike
```

Implementation is in `R/results_appraisal_summary.R`. The older summary key `population_size` retains its geographic/reference meaning for compatibility; use `res_population_cf_total` for the new CF appraisal population display.

6.1 Health outcome units and aggregation

Start here for “what do the health numbers mean?” The input is not a table of observed diagnoses. HM simulates replicants, counts events and divides by the number of replicants. Its source quantities are per-person expectations. HUB matches/interpolates these to synthetic people and calculates CF changes. Summing person-level expectations gives expected counts, which may be fractional.

The authoritative source definitions are in MIAMA-HM `scripts/aggregation.R` (event counts divided by `n.rep`, but death shares are separate ratios), `scripts/sp_hm_join.R` (synthetic-person matching), and `scenario_verification/scen_30to45_2mmets.qmd` (scenario aggregation). HUB calculation details live in `R/results_data_prepare.R`, `R/results_data_plotting_functions.R` and `R/health_outcomes_calculate_halys.R`.

Source fields and their units

Field/family	Meaning in the source	Correct treatment in HUB
dead	Expected deaths per source person in a model cycle, normalized by original HM replicants	Sum across people/cycles for expected deaths; do not describe this as disease incidence
coronary_heart_disease, stroke, diabetes, depression, all_cause_dementia, individual cancer fields, parkinson.s_disease	Expected disease-event contributions per source person-cycle	Sum for expected disease cases/events, not the number of unique people with any disease
unhealthy	Net transition into diseased-or-dead state per source person-cycle	Accumulate to reconstruct healthy state; negative values can represent recovery/remission, not invalid incidence
depression_remission	Modelled depression remission/exits per person-cycle	Use in prevalence reconstruction; not a disease-benefit headline by itself

Field/family	Meaning in the source	Correct treatment in HUB
<code>death_share_*</code>	Proportion of deaths occurring with each disease, conditional on deaths in that cycle	Use in prevalence/HALY reconstruction; never sum as deaths or cases
<code>ly, hly, yrs_*</code> in other/overall HM artifacts	Life, healthy-life or disease-state years per modelled person	Not the current Tab 5 aggregation input; current LY/HLY are reconstructed from cycle states, avoiding summing pre-aggregated horizon totals again
<code>haly</code> , added by HUB	Health-adjusted person-years contributed in one model cycle	Sum across people and years; benefits have the opposite sign to prevented deaths/cases
<code>d_*</code> , added by lookup/HUB	CF minus REF, in the units of the corresponding field	Retain that sign internally; convert to benefit direction only for presentation

The UI catalogue maps mortality to **dead**; CVD to coronary heart disease plus stroke; cancers to the ten configured cancer streams; other options to their named underlying stream. The headline disease total counts each configured raw stream once. **Do not sum displayed CVD with IHD/stroke, or cancers with their displayed subtypes:** those selections overlap. Likewise deaths, cases and HALYs are different quantities and must not be added into one generic health total.

Aggregation formulas Let $r[i,t]$ and $c[i,t]$ be REF/CF person-cycle contributions and w the configured person weight. The current model uses annual cycles.

```

annual REF expected total[t] = sum_i(w * r[i,t])
annual CF expected total[t]  = sum_i(w * c[i,t])
period total                  = sum over included positive model cycles
raw delta                     = CF total - REF total

deaths/cases prevented        = REF total - CF total
life-years/HALYs gained       = CF total - REF total
relative improvement (%)       = 100 * benefit / REF total
benefit per 100,000 people    = 100000 * benefit / represented population

```

Compute ratios **after summing their numerators and denominators**, not by averaging individual percentages. A zero REF denominator gives an unavailable percentage, not zero benefit. A period total per 100,000 is **not an incidence rate per person-year**; no at-risk person-time denominator is calculated here. Annual plots use represented people with source rows in that cycle. Cumulative plots and overall totals use the cohort denominator (maximum population across cycles within each baseline age/sex stratum), not a sum of repeated person-years or a shrinking final-cycle denominator. Age groups use baseline age.

For modelled year contributions, transformation comes before aggregation:

```

LY[i,t]  = 1 - cumulative deaths[i,0:t]
HLY[i,t] = 1 - cumulative net unhealth[i,0:t]
HALY[i,t] = LY[i,t] * (1 - residual pYLD[i,t])
           * (1 - sum_d(prevalence_among_alive[d,i,t] * adjusted_DW[d,i,t]))

```

HUB reconstructs prevalence from disease inflows and deaths/remission. See the HALY section above for details. Cycle 0 initializes these states but earns no reported life-year/HALY and is not counted as an incident event in Tab 5. This is a discrete annual-cycle convention, not a within-year survival integral. Currently main tables and plots include all available nonzero cycles, while AMAT/headline totals stop at the configured assessment horizon. This is not intrinsically wrong; it can represent full-follow-up plots alongside a shorter appraisal summary. It does require explicit period labels if those differ. Whether all products should instead use one horizon is an open decision; no change to that selection policy has been made in this review. Missing late-life source rows are not newly observed zero-event years.

One record, one appraisal person: user counts are real-person targets and health aggregation uses `person_weight = 1` by default for all datasets. Leeds' historical 163.552 source expansion is retained only as `source_person_weight`. Sampling, eligible donor pools and replacement rules are unchanged. Absolute

Leeds health totals are consequently 1/163.552 of the previous weighted totals for identical sampled records; percentages and per-100,000 results are unchanged. Recreate `cfg` and rerun the appraisal: existing result objects and previously saved configurations retain their old weight. Explicit custom `person_weight` overrides remain supported but depart from the default real-person contract.

Labels and mode attribution Use “expected deaths”, “expected disease cases”, “life-years gained” and “HALYs gained” as appropriate. For a mixed percentage overview use “health improvement from reference (%)”, since fewer deaths/cases but more HALYs are beneficial. Do not label the combined panel “incidence rates by disease”. Unlike outcome units must remain separate even when plotted together.

Mode-specific health effects are allocations of the combined person’s health delta using signed mode-specific MMET changes. They are not separately modelled REF/CF burdens. Their absolute REF/CF values and relative percentages are therefore unavailable; attributable changes can be reported as counts or per 100,000. Opposing mode effects can produce negative allocations or shares above 100%, while the allocated effects sum to the combined effect.

Inspect actual source values Run `inst/workflows/dev_health_outcome_audit.R` with HUB loaded. It writes the outcome catalogue, numeric field ranges/missingness and row counts by cycle. By default it reads the configured death-share cycle source. To inspect the calculated run instead, first set:

```
audit_health_data <- hub$counterfactual_data$health_outcomes
source("inst/workflows/dev_health_outcome_audit.R")
```

The 2026-09-07 Leeds source audit found 192,116 rows covering cycles 0-40: 5,000 rows in cycles 0-10, falling to 3,988 in cycle 40. `dead` ranged from 0 to 0.297, `diabetes` from 0 to 0.375, `unhealthy` from -0.067 to 0.789; death shares ranged within 0-1. No non-finite values occurred in these fields. These are descriptive checks, not independent validation of HM probabilities.

Remaining review points: aggregation currently uses `na.rm = TRUE` and can hide missing values in future datasets; custom outcome definitions must follow the supported event/year semantics; source follow-up coverage and the separate real/synthetic unit contract need continued review. The audit should be rerun when HM data are replaced.

`prepare_results_data()` prepares compact Step 8 outputs for MIAMA-UI. The expensive reference, counterfactual, and HM steps run before this function; interactive Tab 5 filters operate on compact result tables and do not rerun the health model.

Cycle 0 is the without-scheme baseline state and is excluded from presented impact totals and timelines. Cycle 1 is the first modelled year. Raw HM `d_*` columns retain the technical convention `counterfactual - reference`. Benefit-oriented fields use `reference - counterfactual` for adverse outcomes (deaths and disease cases) and `counterfactual - reference` for HALYs, so positive values consistently indicate a health gain. Legacy column names such as `prevented_value` are retained in the compact result contract even when the selected positive outcome is more naturally described as HALYs gained.

One selected appraisal person record represents one real person: `cfg$population$person_weight` defaults to 1 for sample, Leeds and full datasets. Entering 1,000 users still samples 1,000 records; the sampling pool, probabilities and feasibility rules are unchanged. Geographic expansion is retained only in `cfg$population$source_person_weight`, not applied to health totals or denominators. Trip counts remain row targets and totals without an extra expansion factor.

For identical records, absolute Leeds health totals are 1/163.552 of the previous weighted totals; relative effects and per-100,000 results are unchanged. Recreate the configuration and rerun the appraisal after updating HUB. Previously saved configurations/results retain their old weight. Explicit custom person-weight overrides remain supported but depart from the default real-person contract. Source-derived default counts describe an example appraisal, not the entire LAD; the full geographic population estimate in Tab 1 remains contextual information.

Tab 5 health outcome choices

The exported helper returns the stable configured choices without loading HM data:

```
outcome_options <- MIAMAHUB::get_health_outcome_options(hub_cfg)

# Named vector suitable for a Shiny checkbox/select control.
outcome_choices <- stats::setNames(
  outcome_options$outcome,
  outcome_options$label
)
default_outcomes <- outcome_options$outcome[outcome_options$default]
```

For development or source validation, pass a health data frame, Arrow dataset, parquet directory, or character vector of column names. Parquet inspection reads only the schema and does not collect rows:

```
checked_options <- MIAMAHUB::get_health_outcome_options(
  cfg = hub_cfg,
  health_data = health_outcomes,
  available_only = TRUE
)
```

`available` confirms that all configured reference columns exist. `delta_available` and `counterfactual_available` separately report whether the matching `d_*` and `*_cf` columns have already been produced. The equivalent R6 call is `hub$get_health_outcome_options()`. The `direction` and `unit` columns distinguish adverse incidence outcomes from positive HALY outcomes; UI code should use those fields rather than assuming every selected outcome is a prevented case.

Shared UI option catalogues

Stable control IDs and labels live in `miama_default_config()`, rather than in plot functions or server conditionals. The catalogues cover age groups, gender, PA and trip-distance categories, trip purpose, setup/result modes, timeframes, temporal and population aggregation, impact presentation, plot metric, and annual versus cumulative timelines.

```
MIAMAHUB::get_ui_options() # discover available catalogues
age_choices <- MIAMAHUB::get_ui_options("age_groups", hub_cfg)
metric_choices <- MIAMAHUB::get_ui_options("metric", hub_cfg)

result_options <- MIAMAHUB::get_results_options(hub_cfg)
result_options$assessment_period_years
result_options$impact_type
```

Each option table has `value`, `label`, `order`, and `default` columns. UI can construct a named Shiny choices vector with `setNames(options$value, options$label)`. Equivalent R6 methods are `hub$get_ui_options()` and `hub$get_results_options()`.

The canonical assessment period is `cfg$results$assessment_period_years` (currently 40 years), available through `get_assessment_period(cfg)` or `hub$get_assessment_period()`. UI titles, summary cards, total aggregation labels, AMAT exports, and report text should all use this value rather than embedding a number.

Tab 5 UI integration lifecycle

Tab 5 has one expensive calculation boundary and a separate lightweight presentation layer. UI should call `Hub$build_results(profile)` once after all counterfactual inputs have been submitted. That method builds counterfactual rows, converts changed weekly activity to MMET exposure, applies the HM cycle lookup, and prepares compact result tables:

```
mdata[["result"]] <- mdata[["hub"]]$build_results(
  profile = mdata[["profile"]],
  seed = 1L
)
```

Do not call `build_results()` again when a user changes only a Tab 5 display control. Keep `mdata[["result"]]` in session state and filter that compact object reactively. The large synthetic-population and HM calculations do not need to rerun.

The primary returned objects are:

Object	Purpose
<code>result\$highlights</code>	Four-row, display-ready headline table for the Highlights Card.
<code>result\$results_data\$headline_metrics</code>	Internal named values supporting the four unfiltered assessment-period highlights.
<code>result\$results_data\$results_table</code>	Table aggregated according to the profile's initial Tab 5 selections. Useful for exports and initial tables.
<code>result\$plot_data\$health_cube</code>	Canonical interactive health source, grouped by outcome, cycle, age group, gender, and mode. It contains <code>all_modes</code> rows and attributed mode rows.
<code>result\$plot_data\$mode_attribution</code>	Compact audit table of mode-specific MMET changes and their shares of the net MMET change.
<code>result\$plot_data\$trip_mode_distribution</code>	Reference/counterfactual trip-record totals and shares by broad mode.
<code>result\$plot_data\$spreads</code>	Reference/counterfactual spread payloads described in the Tab 3/4 section.

`result$plot_data` and `result$results_data$plot_data` refer to the same compact plot payload. HUB plotting and filtering functions expect the enclosing `result$results_data` object, not the bare `health_cube` data frame.

Headline health outcomes After `build_results()`, UI can populate the four highlight figures with one method call:

```
highlights <- mdata[["hub"]]$get_results_highlights()
# The same table is returned directly as mdata[["result"]]$highlights.
# Stateless equivalent: get_results_highlights(mdata[["result"]]$results_data)
```

The returned rows are `premature_deaths_prevented`, `life_years_saved`, `halys_gained`, and `disease_cases_prevented`, with display labels, units, assessment period, and availability status. These totals intentionally ignore interactive Tab 5 filters. The disease total sums each underlying HM incidence stream once; presentation composites such as CVD/all cancers and their subtypes are not double-counted. It is a total of prevented disease events, not unique people.

Tab 5 profile fields and function arguments

Profile/UI field	HUB plotting argument or behavior
<code>res_outcomes</code>	<code>outcomes</code> ; filters health outcome IDs.
<code>res_age_groups</code>	<code>age_groups</code> ; filters the five result age strata.
<code>res_gender</code>	<code>gender</code> ; filters <code>male</code> / <code>female</code> .
<code>res_temp_aggregation</code>	Selects total versus timeline presentation. <code>res_aggregation</code> is accepted as a legacy/internal alias.
<code>res_pop_aggregation</code>	Selects <code>group_by</code> = <code>"none"</code> , <code>"age_group"</code> , <code>"gender"</code> , or <code>"mode"</code> .
<code>res_impact_type</code>	<code>impact_type</code> = <code>"attributable"</code> or <code>"cf_vs_ref"</code> .
<code>res_modes_filter</code>	<code>modes</code> for health and trip plots. <code>all_modes</code> selects canonical combined health impacts; <code>walking</code> , <code>cycling</code> , and other attributed components select the corresponding health allocation.

Two useful presentation choices are not currently separate profile fields, but their canonical values and labels are exposed by `get_ui_options()`:

- `metric`: "prevented", "prevented_per_100000", or "percent_reduction"; each plot function has a documented default.
- `timeline_type`: "cumulative" (default in the timeline plot) or "annual". A future UI toggle can pass this directly without rebuilding results.

The current `res_impact_type` UI label still describes attributable cases as counterfactual minus reference. Internally, raw `delta_value` retains that technical `cf - ref` convention, but all user-facing attributable metrics use `ref - cf`: positive `prevented_value`, `prevented_per_100000`, and `percent_reduction` mean a health gain. UI labels should follow the latter presentation convention.

Recommended reactive plot calls For a simple outcome overview, call the HUB plotting function inside `renderPlot()`. It returns a ggplot object:

```
output[["results_health_overview"]] <- shiny::renderPlot({
  shiny::req(mdata[["result"]])

  MIAMAHUB::results_plot_health_overview(
    results_data = mdata[["result"]]$results_data,
    outcomes = input[["res_outcomes"]],
    age_groups = input[["res_age_groups"]],
    gender = input[["res_gender"]],
    modes = input[["res_modes_filter"]],
    impact_type = input[["res_impact_type"]],
    metric = "percent_reduction"
  )
})
```

For the primary cumulative timeline:

```
output[["results_health_timeline"]] <- shiny::renderPlot({
  shiny::req(mdata[["result"]])

  MIAMAHUB::results_plot_health_timeline(
    results_data = mdata[["result"]]$results_data,
    outcomes = input[["res_outcomes"]],
    age_groups = input[["res_age_groups"]],
    gender = input[["res_gender"]],
    modes = input[["res_modes_filter"]],
    impact_type = "attributable",
    metric = "prevented_per_100000",
    timeline_type = "cumulative"
  )
})
```

For advanced age or gender views, use the detailed plot only when `res_pop_aggregation` requests a stratum; use the overview plot for "total":

```
shiny::req(input[["res_pop_aggregation"]] %in% c("age_group", "gender"))
group_by <- switch(
  input[["res_pop_aggregation"]],
  age_group = "age_group",
  gender = "gender"
)

MIAMAHUB::results_plot_health_impacts(
  results_data = mdata[["result"]]$results_data,
  outcomes = input[["res_outcomes"]],
  age_groups = input[["res_age_groups"]],
  gender = input[["res_gender"]],
  modes = input[["res_modes_filter"]],
  group_by = group_by,
  metric = "prevented_per_100000"
)
```

For trip-record mode shares:

```

MIAMAHUB::results_plot_trip_mode_distribution(
  results_data = mdata[["result"]]$results_data,
  modes = input[["res_modes_filter"]],
  value = "proportion"
)

```

Use `plotly::ggplotly(plot, tooltip = "text")` if Tab 5 requires Plotly interaction. Every HUB plot includes a formatted `tooltip_text` aesthetic with semantic labels, context, units, and rounded values. Restricting Plotly to "text" prevents raw internal field names from appearing. HUB does not require Plotly and returns ordinary ggplot objects so UI controls the rendering technology.

UI-owned plotting alternative MIAMA-UI may reproduce the plots rather than call HUB's ggplot functions. In that case, use the exported filtering function so aggregation and sign conventions remain identical:

```

plot_df <- MIAMAHUB::results_filter_health_data(
  results_data = mdata[["result"]]$results_data,
  outcomes = input[["res_outcomes"]],
  age_groups = input[["res_age_groups"]],
  gender = input[["res_gender"]],
  modes = input[["res_modes_filter"]],
  aggregation = "timeline",
  group_by = "outcome",
  timeline_type = "cumulative"
)

```

The returned data include `ref_value`, `cf_value`, internal `delta_value`, `population`, `prevented_value`, `prevented_per_100000`, `percent_reduction`, and display labels. UI should plot the benefit-oriented columns unless it is explicitly presenting both reference and counterfactual levels.

Plot/filter calls do not modify the profile and return no profile fields. The UI may store Tab 5 selections in their existing profile `input_value` fields, but changing those display controls does not require a HUB round trip or a new health-model run.

Mode selections have two aggregation behaviors. For `group_by = "mode"`, HUB returns one attributed series per selected mode; `modes = NULL` or `modes = "all_modes"` displays every available attributed mode separately. For outcome, age, gender, and timeline views, multiple selected modes are summed into one `selected_modes` series; selecting one mode returns that mode alone. Leaving `modes = NULL` in those combined views uses the canonical `all_modes` health result.

Reporting units in the current UI The age plot and timeline now pass `modes = NULL`: they show combined appraisal health outcomes filtered by age/sex/outcome. Mode selections apply to the separate mode-impact and trip plots only. The main overview remains fixed and unfiltered. Timeline Presentation affects only the timeline, not the age/mode bars.

Reporting unit	Benefit/difference plots	Separate REF/CF timeline
Absolute values	Deaths/cases prevented or HALYs gained	Expected deaths/cases or HALYs in each scenario
Values per 100,000 people	Benefit divided by assessed population, times 100,000	Each scenario value divided by assessed population, times 100,000
Percentage improvement	100 times benefit divided by REF outcome total	Unsupported; use the benefit view or another unit

Positive benefit means REF minus CF for deaths/cases and CF minus REF for HALYs. Age/mode bars cover the included assessment cycles; timelines are cumulative by default. Cumulative per-100,000 values use the represented cohort denominator, not a denominator shrinking as late-cycle source rows disappear. This is not an incidence rate per person-year. Outcome panels label their units explicitly and unlike outcomes are not added together. Benefit timelines display outcome lines on one shared scale; separate REF/CF timelines retain their outcome panels. Axes use short labels, with the period and detailed outcome units in subtitles, captions and tooltips. A shared scale does not make HALYs and event counts interchangeable or additive.

Mode-attributed percentages have no separate REF denominator. HUB now displays an explanatory unavailable view instead of silently switching to absolute values. Separate REF/CF plots likewise explain that percentage improvement requires a difference view. UI captions come from the actual generated plot; filter-only changes do not rerun sampling or the health model.

The central UI plotting contract is returned directly by `Hub$build_results()` as `result$plot_data` and is also available as `result$results_data$plot_data`. Both names refer to the same compact object in the R session; HUB does not recalculate or copy the underlying values.

The payload intentionally contains shared source tables rather than one data frame per plot:

- `health_cube` is aggregated by outcome, model cycle, age group, gender, and mode. It contains canonical `all_modes` rows plus mode-attributed delta rows and is the comprehensive source for the health overview, timeline, age, gender, and mode plots.
- `mode_attribution` is the compact audit table behind the mode split. It reports changed individuals, MMET deltas, and net MMET shares by mode.
- `trip_mode_distribution` contains reference and counterfactual trip-record totals and proportions for Walking, Cycling, Public transport, Driving, and Other. It remains separate because its rows represent travel modes and scenarios, whereas `health_cube` rows represent health-model strata.
- `spreads` contains compact reference/counterfactual spread-bar data produced by `build_results()` for the population, trip, and physical-activity spread views.
- `health_overview` and `health_timeline` remain convenience tables for current request defaults and exports. UI does not need them for interactive plotting when it uses `health_cube`.

Combining health and trip data into one data frame would create sparse columns and ambiguous units. Producing one table per plot would repeat the same health values and allow the views to drift apart. Two canonical analytical tables, plus the distinct spread payload, preserve each table's grain while supporting all current Tab 5 interactions.

```
result <- hub$build_results(profile)

# Direct reactive filtering in UI.
health_cube <- result$plot_data$health_cube
trip_modes <- result$plot_data$trip_mode_distribution

# Or use HUB's filter and plotting functions against the enclosing object.
health_by_age <- results_filter_health_data(
  result$results_data,
  outcomes = c("mortality", "ihd", "stroke"),
  modes = "all_modes",
  aggregation = "total",
  group_by = "age_group"
)
plot <- results_plot_health_impacts(
  result$results_data,
  outcomes = c("mortality", "ihd", "stroke"),
  modes = "all_modes",
  group_by = "age_group"
)

# Allocate combined health deltas to the active modes that supplied the MMET
# change. This contains attributable changes, not mode-specific reference and
# counterfactual disease totals.
plot_by_mode <- results_plot_health_impacts(
  result$results_data,
  outcomes = c("mortality", "ihd", "stroke"),
  modes = c("walking", "cycling"),
  group_by = "mode",
  metric = "prevented"
)
```

Mode attribution contract The health model is run once for the combined counterfactual exposure. HUB then allocates each individual's resulting health delta using that individual's signed MMET components: `mode share = mode MMET delta / total MMET delta` and `mode health delta = combined health delta * mode share`. Walking, cycling, other activity, and any numerical residual are retained so their attributed health deltas reconcile exactly to the canonical `all_modes` result.

This is an attribution of the combined model result, not a separate health-model run for each mode. Shares can be negative or greater than one when one mode offsets another. Mode rows therefore contain attributable deltas only; `ref_value` and `cf_value` are NA. Direct reference-versus-counterfactual totals and percentage reductions require the canonical `all_modes` rows. Headline figures and AMAT exports also remain canonical all-mode totals to avoid double counting.

For a lean Shiny reactive, UI can retain `result$results_data` (or only `result$plot_data` when it performs its own filtering) instead of retaining the large `reference_data` and `counterfactual_data` objects returned for current development inspection.

Tab 5 result exports

HUB provides one static export bundle and file writers so MIAMA-UI does not need to reimplement result semantics. The current app builds this bundle once, immediately after `build_results()`, and stores it as `mdata[["results_export"]]`:

```
mdata[["results"]] <- hub$build_results(mdata[["profile"]])
mdata[["results_export"]] <- hub$build_results_exports()
```

With no explicit arguments, the bundle uses the initial `results_request` stored in `results_data`. It is not rebuilt when the advanced Tab 5 filter panel changes. Those interactive `results_filter_*` controls currently update only the on-screen plots. Consequently, downloads are reproducible snapshots of the result settings present when results were built, but the primary CSV is not a raw dump of every `health_cube` row. Developers can inspect the comprehensive, filterable source tables under `results_data$plot_data`.

`build_results_exports()` still accepts explicit outcome, age, gender, mode, aggregation, grouping, timeline, impact, and metric arguments for development or possible future filtered exports. Passing those arguments creates a new static selection snapshot; it does not rerun the counterfactual or health model.

The equivalent stateless function is `prepare_results_exports(results_data, profile, cfg, ...)`. Its return value contains:

Object	Draft export content
<code>metadata</code>	Appraisal name, geography, modes, UI version, population scaling, and result conventions.
<code>filters</code>	Initial results-request selections captured when the static bundle was assembled.
<code>headline_metrics</code>	Mortality, disease, and cumulative life-year headline fields.
<code>results_table</code>	Health result table grouped according to the bundle's initial selection snapshot.
<code>timeline_annual / timeline_cumulative</code>	Both timeline representations using the same outcome/population/mode snapshot.
<code>trip_mode_distribution</code>	Reference/counterfactual trip records and mode shares for the snapshot's modes.
<code>assumptions</code>	Sign conventions, population scaling, cycle handling, and current limitations.
<code>amat_inputs</code>	Explicitly provisional field/value mapping pending the agreed AMAT schema.
<code>amat_health_timeline</code>	Annual ref/cf values, technical differences, benefit-oriented differences, and cumulative differences for deaths, diseases, LY, HLY, and HALY.
<code>amat_health_summary</code>	Final cumulative row for every AMAT health measure.
<code>report</code>	Report-ready title, summary text, methods, metadata, static selection table, and assumptions.
<code>plots</code>	Six static ggplot objects built when the bundle is assembled, including a mode-attributed health plot.

Available writers are:

```

write_results_csv(results_exports, "results.csv")
write_results_xlsx(results_exports, "results.xlsx")
write_results_plot_pngs(results_exports, "plots", dpi = 300)
write_results_plots_zip(results_exports, "plots.zip", dpi = 300)
write_results_amat_csv(results_exports, "amat_health_timeline_DRAFT.csv")
write_results_report(results_exports, "report.md", format = "markdown")
write_results_report(results_exports, "report.docx", format = "docx")
write_results_report(results_exports, "report.pdf", format = "pdf")

```

The Excel workbook contains separate sheets for the displayed results, headline metrics, annual and cumulative timelines, trip modes, metadata, filters, assumptions, AMAT metadata, and AMAT health timeline/summary sheets. The plot package contains 300-dpi PNG files for health overview, health timeline, health by age, health by gender, health by mode, and trip mode distribution, plus `plot_manifest.csv`. Their semantic titles, units, and captions adapt to metric, impact type, annual/cumulative presentation, and grouping. They do not currently enumerate every selection snapshot filter in the title or caption; the separate `filters` table records that context.

Appraisal report: inputs, results and assumptions

`write_results_report(results_exports, "report.html", format = "html")` creates a self-contained, printable report without Pandoc or LaTeX. Word still requires Pandoc; the HUB PDF writer additionally requires a PDF engine. The UI report card offers HTML and Word rather than a placeholder PDF download.

The report covers the appraisal metadata, submitted settings, effective calculation inputs (including accepted population/trip values), realized travel totals, health results and annual impacts, completion assumptions with provenance, and configured sampling/model parameters. The XLSX export includes corresponding `Calculation_inputs`, `Submitted_settings`, `Completion_assumptions` and `Model_parameters` sheets. Tables are not silently truncated at 100 rows.

`R/results_appraisal_record.R` owns the record and report sections. The calculation record is captured by `prepare_results_data()`; `Hub$build_results_data()` adds submitted settings, effective assumptions and descriptive metadata from the completed run. Older results objects remain exportable but missing sections are marked as not recorded. Rebuild results to obtain the full record.

Interpret the sections separately: a saved setting can be inactive; configuration contains defaults which explicit inputs may override; requested percentages do not prove an identical realized split. This is a parameter record, not a full sampling-event audit or uncertainty analysis. Raw people/trips, profile `additional_data` and local data paths are not exported. Plot embedding, detailed realized donor/fallback diagnostics and final branding remain follow-up work. The existing export selection snapshot is unchanged and is listed in the report; later Tab 5 plot-filter edits do not change it. Headline metrics cover the whole appraisal even if an explicit export selection narrows the health tables.

The AMAT health timeline uses the canonical assessment period configured at `cfg$results$assessment_period_years` (currently 40 years). It implements the MIAMA-HM definitions $LY[t] = 1 - \text{cumulative deaths}[t]$ and $HL[t] = 1 - \text{cumulative unhealth incidence}[t]$. For every measure it exposes both `annual_delta_cf_minus_ref` and `cumulative_delta_cf_minus_ref`, plus benefit-oriented columns where positive consistently means improvement. For deaths and diseases, benefit is `ref - cf`; for LY, HLY, and HALY it is `cf - ref`. HALYs use the reconstructed-prevalence and disability-weight method described under Counterfactual health outcomes. AMAT field names and the required disease subset remain provisional until the formal specification is supplied.

The same health contract can be prepared without the rest of the export bundle:

```

amat <- prepare_results_amat_outputs(
  result$results_data,
  horizon_years = 40
)
amat$timeline
amat$summary

```

Minimal Shiny handlers can delegate directly to HUB:

```

output[["download_results_csv"]] <- shiny::downloadHandler(
  filename = function() "miama-results.csv",
  content = function(file) {
    MIAMAHUB::write_results_csv(mdata[["results_export"]], file)
  }
)

output[["download_results_xlsx"]] <- shiny::downloadHandler(
  filename = function() "miama-results.xlsx",
  content = function(file) {
    MIAMAHUB::write_results_xlsx(mdata[["results_export"]], file)
  }
)

output[["download_plots"]] <- shiny::downloadHandler(
  filename = function() "miama-plots.zip",
  content = function(file) {
    MIAMAHUB::write_results_plots_zip(mdata[["results_export"]], file)
  }
)

output[["download_amat"]] <- shiny::downloadHandler(
  filename = function() "miama-amat-inputs-DRAFT.csv",
  content = function(file) {
    MIAMAHUB::write_results_amat_csv(mdata[["results_export"]], file)
  }
)

output[["download_report_docx"]] <- shiny::downloadHandler(
  filename = function() "miama-report.docx",
  content = function(file) {
    MIAMAHUB::write_results_report(
      mdata[["results_export"]], file, format = "docx"
    )
  }
)

```

Step 9 of `inst/workflows/dev_workflow.R` builds and displays the same in-memory static export bundle as MIAMA-UI. Run `inst/workflows/dev_results_exports.R` afterward, in the same R session, to write all draft artifacts locally.

`results_filter_health_data()` is the non-plot interface UI can use to obtain a filtered data frame. Its arguments mirror the Tab 5 controls:

```

plot_df <- results_filter_health_data(
  results_data,
  outcomes = c("mortality", "ihd", "stroke"),
  age_groups = c("age_40_49", "age_50_59", "age_60_plus"),
  gender = c("male", "female"),
  modes = "all_modes",
  aggregation = "timeline",
  group_by = "age_group",
  timeline_type = "cumulative"
)

```

Development plotting functions consume the same compact data. The intended Tab 5 structure is deliberately limited to two core and four advanced views:

- Core 1, `results_plot_health_overview()`: cumulative health impact by outcome, shown as prevented outcomes, prevented outcomes per 100,000 residents, or percentage reduction. Direct reference/counterfactual totals remain an optional diagnostic because their lines often overlap at realistic effects.
- Core 2, `results_plot_health_timeline()`: cumulative impact from cycle 1 by default, with annual impact available through `timeline_type = "annual"`.
- Advanced 1 and 2, `results_plot_health_impacts()`: cumulative health impacts split by age group or by gender and faceted by outcome.
- Advanced 3, `results_plot_health_impacts(group_by = "mode")`: combined health deltas attributed to walking, cycling, and other activity according to their individual-level MMET contributions.
- Advanced 4, `results_plot_trip_mode_distribution()`: reference and counterfactual trip-record shares or weekly row totals by mode.

Each function provides data-aware defaults for `title`, `subtitle`, `caption`, `x_label`, and `y_label`; callers can override any of them. `NULL` uses the semantic default and `NA_character_` suppresses the label. Units are selected from the plot request and data: health plots distinguish deaths, disease cases, and mixed health outcomes; total plots are cumulative across the available model cycles; timeline plots are annual or cumulative according to `timeline_type`; trip shares are displayed as percentages; and trip totals are trip records per reference week. Label overrides change presentation only and never transform the numeric values. The same semantics are used in the embedded `tooltip_text` aesthetic. Health tooltips identify the outcome, scenario or grouping, model year where relevant, and requested metric; trip tooltips identify mode, scenario, and row count or share.

For attributable health plots, `prevented_value = reference - counterfactual` and `percent_reduction = 100 * (reference - counterfactual) / reference`. `prevented_per_100000` divides prevented outcomes by the represented population. Positive values therefore indicate a health gain. Plot captions state this sign convention and define reference as without scheme and counterfactual as with scheme.

These ggplot functions currently live in HUB so the result contract and example presentation can be developed together. MIAMA-UI can call them directly or reproduce their styling from the returned data frames. Omit `modes`, or use `modes = "all_modes"`, for the canonical combined health result. Pass `modes = c("walking", "cycling", "ebiking", "pt")` for attributed health deltas. Because mode rows do not have separate reference denominators, mode-specific requests use prevented counts or prevented counts per 100,000 rather than direct scenario totals or percentage reduction.

7. Data packaging, deployment, and performance

Synthetic population parquet conversion

Runtime synthpop sources are parquet datasets. Legacy `.dta` files are only supported as optional local conversion inputs and are not required by the HUB pipeline.

A development helper, `convert_synthpop_to_parquet()`, is available for this one-time local conversion.

`miama_paths()` resolves to `SPindivd_CensusNTSALS_dev_parquet` when present, otherwise `SPindivd_CensusNTSALS_parquet` with the equivalent trip paths for `SPtrip_CensusNTSALS`. `load_reference_sources()` then prefilters by geography before collecting data into R.

The geography option lookup is a second small one-time/cacheable artifact. It is created by `build_geo_lookup(overwrite = TRUE)` or lazily by `get_geo_levels()` / `get_geo_options()`, and is stored at `data/lookup/geo_options.rds`. For package deployments, a copy of the full-derived lookup is stored under `inst/extdata/data/lookup/geo_options.rds`.

Development integration backlog

The current `appraisal_inputs` setup is deliberately temporary and dev-focused. Later work should:

1. replace the mock helper with direct integration from MIAMA-UI
2. decide the runtime interface between UI and HUB (likely via package calls or Shiny server integration, not necessarily HTTP)
3. strengthen validation of required fields and allowed values
4. expand mapping from raw UI inputs into explicit reference, counterfactual, and results specifications
5. revisit the optimal timing of HM joins relative to counterfactual generation
6. keep `DESCRIPTION` updated whenever new package dependencies are introduced

Runtime data profiles and locations

Dataset scope is a runtime/deployment setting, not an appraisal input. Set it before creating the HUB configuration:

```
# Project .Renvirom (restart R after editing)
MIAMA_DATASET_SIZE=sample
```

MIAMA-HUB supports three coherent runtime data profiles:

- **sample** is the default, very small packaged cross-geography fixture. It is useful for fast automated tests but is too sparse for realistic appraisal inspection; Leeds contains only 16 people.
- **leeds** is a packaged Leeds-only profile with 5,000 people, 83,534 trips, and aligned overall and death-share cycle HM outcomes. It is the recommended profile for local UI development, demonstrations, and published test apps.
- **full** reads full external synthpop and MIAMA-HM data and is intended for production-scale analysis rather than deployment inside the package.

Both packaged profiles ignore `MIAMA_DATA_ROOT`. This prevents external synthpop rows from being combined with unrelated packaged HM outcomes. Explicit `cfg$sources` overrides remain available for controlled development tests.

Select the Leeds profile either through the process environment or directly:

```
MIAMA_DATASET_SIZE=leeds
```

```
cfg <- MIAMAHUB::miami_default_config(dataset_size = "leeds")
```

The Leeds profile only offers Leeds (E08000035) as an LAD option. Its 5,000 people are a reproducible simple random sample from all 40,888 Leeds synthpop rows. The geographic expansion factor is 163.552 (20 / **sample fraction**), describing an 817,760-person source-population estimate. This is metadata only: the default appraisal has 5,000 people and health results use unit person weight. Sampling provenance and row counts are stored in `inst/extdata/data/profiles/leeds/profile.rds`.

Manchester: a separate 10,000-person profile `miami_default_config(dataset_size = "manchester")` selects Manchester LAD (E08000003), not Greater Manchester. Its independent files live under `inst/extdata/data/profiles/`. Leeds remains unchanged. Both profiles use their own aligned people, trips and health outcomes, and share only common health lookup/parameter files. No local MIAMA-HM checkout is needed at runtime.

The sample contains 10,000 distinct people drawn without replacement from the 27,864 available Manchester source records (seed 20260908). Their existing trips, overall health outcomes and death-share cycle outcomes are matched by `census_id`; missing health histories abort the build. Geographic expansion is metadata only: appraisal person weight remains 1, including for Manchester.

To reproduce from the full local sources, run from HUB:

```
MIAMA_PROFILE_ID=manchester Rscript --vanilla inst/workflows/dev_build_packaged_lad_profile.R
Rscript --vanilla inst/workflows/dev_build_packaged_data_manifest.R
```

The builder refuses to replace an existing profile unless `MIAMA_PROFILE_OVERWRITE=true`. Source locations are controlled by `MIAMA_DATA_ROOT` (default HUB/data) and `MIAMA_HM_ROOT` (default sibling MIAMA-HM). Sample size, seed and LAD can be overridden with `MIAMA_PROFILE_SAMPLE_N`, `MIAMA_PROFILE_SEED` and `MIAMA_PROFILE_GEO_ID`. The original Leeds builder is a compatibility entry point; its existing `MIAMA_LEEDS_*` options still work.

For the development workflow, set `MIAMA_DEV_DATASET_SIZE=manchester`. For UI, construct its config with `dataset_size = "manchester"`; an explicit `"leeds"` argument overrides environment settings. This is a deployment/profile choice, with a combined Leeds/Manchester selector supported through `get_packaged_geo_options(cfg)` and `select_geographic_profile(cfg, geo_id)`. The matching UI branch uses these helpers and resets the appraisal on city change. Model assumptions are retained while source paths, population metadata and cache locations switch together. Publish the rebuilt HUB package to include the new data; adding this profile does not update a live app.

Full-data testing requires external data that are deliberately excluded from the package and git repository:

```
MIAMA_DATASET_SIZE=full
MIAMA_DATA_ROOT=/absolute/path/to/miama-runtime-data
MIAMA_HM_ROOT=/absolute/path/to/MIAMA-HM
```

MIAMA_DATA_ROOT must contain the full synthpop parquet directories under `synthetic_pop/`. MIAMA_HM_ROOT must contain `health_data/processed/sp_cycle_outcomes_death_share/`. The common death-share MMET lookup is packaged with HUB and may also be supplied from MIAMA-HM. Setting a root only points HUB at existing files; it does not download them.

The UI should construct its shared config once with `MIAMAHUB::miami_default_config()` and should not subsequently hard-code `cfg$workflow$dataset_size`. This lets local `.Renv` files and deployment environment variables select sample, Leeds, or full data without exposing that operational choice in the appraisal profile.

Development scripts that intentionally choose the scope should pass it while constructing the configuration, because source paths are resolved at that time:

```
cfg <- MIAMAHUB::miami_default_config(dataset_size = "leeds")
```

For local development, large data files must not be tracked in git or included in package builds. Set `MIAMA_DATA_ROOT` to an external/local data directory. The legacy project-root `data/` path is ignored by both git and R CMD build, but is no longer selected implicitly.

`inst/extdata/` is different: files under this directory are bundled when MIAMA-HUB is installed as an R package. To keep the package publishable, it contains small runtime/sample data, plus small lookup artifacts that the UI can load without full data access.

Current expected layout:

- full synthetic population files live under `$MIAMA_DATA_ROOT/synthetic_pop/`
- packaged sample synthetic population files live in `MIAMA-HUB/inst/extdata/data/synthetic_pop/`
- the aligned 5,000-person Leeds profile lives under `MIAMA-HUB/inst/extdata/data/profiles/leeds/`
- the packaged `geo_options.rds` is intentionally full-derived because it is small and needed for complete UI geography dropdowns and population labels
- packaged HM sample processed outputs live under `MIAMA-HUB/inst/extdata/data/health_data/`
- packaged HM sample results support includes `sp_cycle_outcomes_sample_death_share/` and `mmet_d_cycle_lookup_death_share/`
- full HM processed outputs are read from MIAMA-HM via `MIAMA_HM_ROOT`

Regenerate the Leeds profile only when the full upstream synthpop or HM data change, or when intentionally changing its sample size or seed:

```
MIAMA_LEEDS_OVERWRITE=true \  
  Rscript --vanilla inst/workflows/dev_build_packaged_leeds_profile.R  
Rscript --vanilla inst/workflows/dev_build_packaged_data_manifest.R
```

The builder defaults to `MIAMA-HUB/data` for current full synthpop parquet and the sibling `MIAMA-HM` repository for full health data. `MIAMA_DATA_ROOT` and `MIAMA_HM_ROOT` can override those source roots. Legacy DTA and RDS synthpop files are not accepted by this workflow.

HM outcome loading follows this hierarchy:

1. cached RDS files in `cfg$cache$dir` when cache is enabled and no census-id prefilter is requested
2. packaged sample parquet directories such as `inst/extdata/data/health_data/sp_overall_outcomes_sample/`
3. external MIAMA-HM parquet directories under `MIAMA_HM_ROOT/health_data/processed/`

Reference and counterfactual individual/trip data use the HM `overall` table, which has one row per individual. A timeline results request does not join the cycle table to trips: death-share cycle outcomes are loaded and filtered by `census_id` only after counterfactual MMET exposure has been created. This avoids an invalid cycle-by-trip row expansion.

During `Hub$build_results()`, source and pre-filter row tables are released once the filtered reference and counterfactual objects have been built. The death-share lookup is evaluated in bounded chunks with MMET-band overlap inside the join, preventing full-data scenarios from materializing all lookup bands for every changed person-cycle row.

The cycle calculation retains reference outcome columns plus `d_* = cf - ref` columns while it runs. Redundant `*_cf` columns are disabled by default because all consumers derive them as `ref + d_*`. After result preparation, the full person-cycle table is removed from `counterfactual_data`; `health_impacts` retains its row/column/size audit and health report. Compact health cubes, AMAT timelines, and

headline metrics live only in `results_data`, avoiding duplicate UI/export payloads. This keeps plot and export functionality while substantially reducing per-session retained memory. Low-level calls to `apply_counterfactual_health_outcomes()` still return the cycle table for model development; pass `include_cf_columns = TRUE` only when explicit convenience columns are needed.

Profiling full-data API performance

[inst/workflows/dev_profile_api_performance.R](#) profiles the two UI-facing data calls against the real MIAMA-UI default profile. It records stage elapsed time, process RSS at API boundaries, retained R6 object sizes, and Rprof call-stack summaries. Run sample mode first, then a full-data LAD after configuring `MIAMA_DATA_ROOT` and `MIAMA_HM_ROOT`:

```
Rscript --no-init-file inst/workflows/dev_profile_api_performance.R

MIAMA_PROFILE_DATASET_SIZE=full \
MIAMA_PROFILE_GEO_LEVEL=lad \
MIAMA_PROFILE_GEO_ID=E08000025 \
Rscript --no-init-file inst/workflows/dev_profile_api_performance.R
```

Set `MIAMA_PROFILE_OUTPUT_DIR` to retain reports at a specific path. The script applies an approximately 10% increase in walking and cycling trip rows so the counterfactual and health stages do real work. `stage_timings.csv` identifies the expensive API stage, `hub_state_sizes.csv` identifies retained large objects, and `rprof_by_total.csv` / `rprof_by_self.csv` identify expensive call stacks. RSS is sampled at stage boundaries; it should not be interpreted as an exact transient peak. `table_inventory.csv` adds row counts, column counts, and retained sizes for the major person, trip, health-cycle, and result tables.

Without `MIAMA_DATA_ROOT`, HUB deliberately uses the packaged sample data. `dataset_size = "full"` fails clearly when only packaged sample synthpop data are available. Callers can also provide explicit `cfg$sources` paths. Longer- term external storage may be a VPS or another managed data service.

`inst/extdata/data/manifest.csv` inventories every packaged artifact with its source, source version, purpose, row count, schema, and checksum. Regenerate it after intentional packaged-data changes with:

```
Rscript --vanilla inst/workflows/dev_build_packaged_data_manifest.R
```

`source_version = "unrecorded"` identifies artifacts for which an authoritative upstream release identifier still needs to be supplied; it is not silently treated as a known version.